COMMONWEALTH OF AUSTRALIA
Copyright Regulations 1969
This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice

Prepared by: [Arun Konagurthu]
Extended by: Taylor Kearney

FIT3155: Advanced Algorithms and Data Structures

Week 7: **Fibonacci heaps**

Faculty of Information Technology, Monash University

What is covered in this lecture?

Fibonacci heaps

Original reference

Michael Fredman and Robert Tarjan, Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms
Journal of the ACM, 34(3) 596-615 (1987). [link]

Source material and recommended reading

- CLRS, Introduction to Algorithms (Chapter 19): Fibonacci heaps [online link]

Priority queues – Self-study slide

An abstract data structure where each element has an associated priority. The element with the highest priority in the queue is served first.

Applications

Priority queues have a variety of important applications:

- Dijkstra's shortest path algorithm
- Prim's algorithm (minimum weight spanning trees)
- Process sheduling
- Sorting

Priority queues (implemented using heaps) – Self-study slide

Recall from FIT2004 that the heap data structure was used to implement priority queues in several applications:

- Heap sort
- Dijkstra's shortest path algorithm
- Prim's algorithm

Recall also that this data structure supports the following operations*:

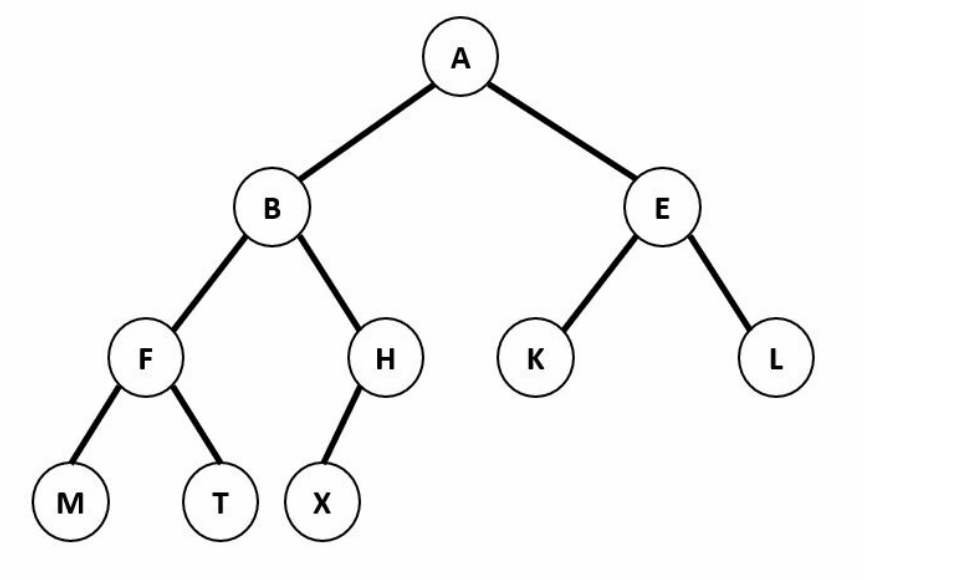
- **`insert`** a new element (containing key w/ payload) into a heap
- identify the **`min`** element in an existing heap
- **`extract-min`** (identify and delete min) element in an existing heap
- **`decrease-key`** of an element in an existing heap

*As with these slides, default heap operations in the rest of the slides are defined over a min-heap. One could alternatively define **`max`**, **`extract-max`**, **`increase-key`** operations on a max-heap.

Binary heaps (recap) – Self-study slide

Properties

- Structure: A binary heap is a complete binary tree, i.e. every level is full except possibly the last.
- Order (min-heap): Every node is less than or equal to its children.



1	2	3	4	5	6	7	8	9	10
A	B	E	F	H	K	L	M	T	X

Binary heaps (recap) – Self-study slide

Operation	Binary heap
`min`	$O(1)$
`extract-min`	$O(\log N)$
`decrease-key`	$O(\log N)$
`delete`	$O(\log N)$
`insert`	$O(\log N)$ worst-case

Mergeable heaps

This week we will learn about a mergeable heap data structure (**`Fibonacci heap`**) that supports the following operations[†]:

- **`insert`**: inserts a new element into the existing heap
- **`min`**: finds the min element in the heap
- **`extract-min`**: finds and deletes the min element in the heap
- **`merge`**: merges two heaps into one
- **`decrease-key`**: decreases the elements key
- **`delete`**: removes an element from the heap

Note, merging two binary heaps requires $O(n)$-time.

[†]Again, the slides define these default heap operations over a min-heap. One could alternatively define **`max`**, **`extract-max`**, **`increase-key`** operations on a max-heap.

Mergeable heaps

This week we will learn about a mergeable heap data structure (**Fibonacci heap**) that supports the following operations[†]:

- **insert**: inserts a new element into the existing heap
- **min**: finds the min element in the heap
- **extract-min**: finds and deletes the min element in the heap
- **merge**: merges two heaps into one
- **decrease-key**: decreases the elements key
- **delete**: removes an element from the heap

Note, merging two binary heaps requires $O(n)$-time.

[†]Again, the slides define these default heap operations over a min-heap. One could alternatively define **max**, **extract-max**, **increase-key** operations on a max-heap.

Motivation for the conceptualization of Fibonacci heaps

- Improve run-time complexity of **Dijkstra's** shortest path algorithm, and other network (directed graph) optimization algorithms.
  - Recall Dijkstra's algorithm from FIT2004?
  - In general, its time complexity is of the form, based on the implementation of the priority queue:

$$O(|V| \times \text{Time}_{\texttt{extract_min}} + |E| \times \text{Time}_{\texttt{decrease_key}}).$$

  - Using binary heaps, its run-time complexity is:

$$O(|V| \log |V| + |E| \log |V|) \approx O(|E| \log |V|).$$

  - Using **Fibonacci heaps** it becomes:

$$O(|V| \log |V| + |E|) \approx O(|E|).$$

  - This is achieved by designing the data structure so that **decrease_key** is $O(1)$-time (**amortized**[‡]), while preserving $O(\log |V|)$ for **extract_min**.

[‡] We will learn what this precisely means in Week 8

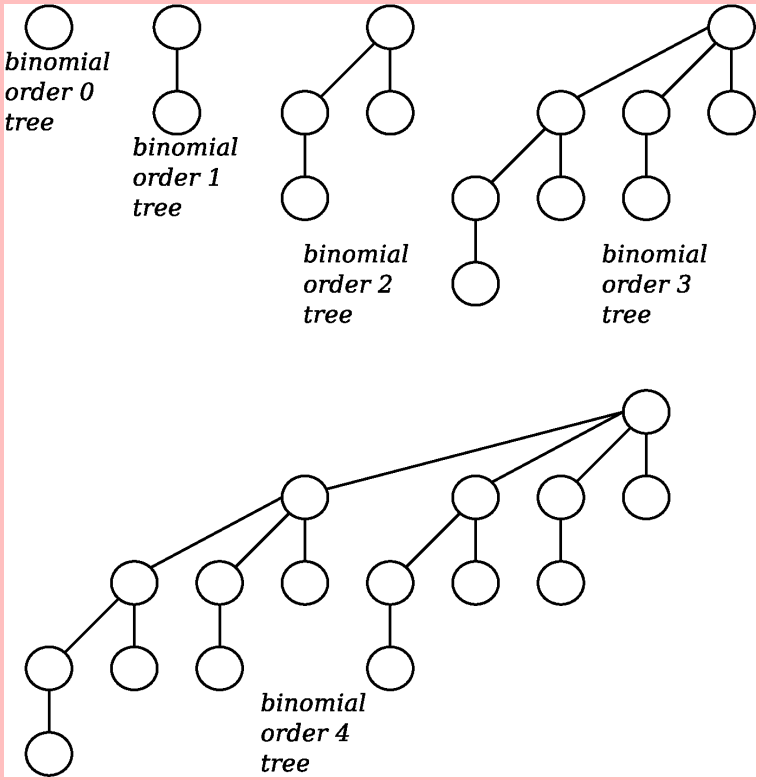
Importance of Fibonacci heaps in the Data Structures and Algorithms literature

- They hold considerable **theoretical** significance in the study of **data structures and algorithmic analyses**.
- They provide computational **lower bounds** maintaining a priority queues and this thinking had influenced the design of other data structures since.
- It introduced **lazy evaluation** in data structure literature.
  - This is in contrast to a precursor data structures, e.g. binomial heaps, that always **eagerly** maintains its strict structure (see next slide).
- It championed **amortized analysis** to the field (which we will learn in more detail next week).

Motivation: Bionomial heap is an 'eager' data structure – not examinable

A binomial **heap** is a collection of binomial **trees** such that:

- Heap property: Each binomial tree in the set satisfies the heap property – i.e., each tree-node's key is $\leq$ its children's keys.
- Structure: There is **at most** one (i.e. either 0 or 1) binomial tree of any given order in that set.



Binomial trees are defined recursively as follows:

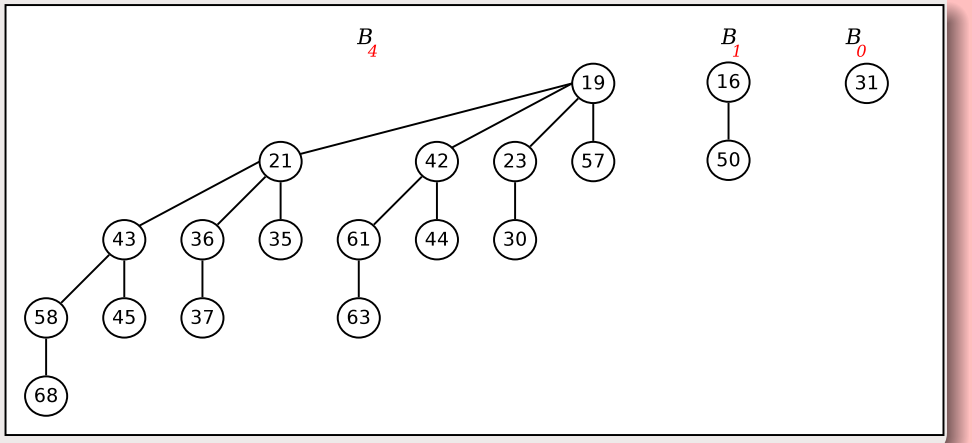
- The binomial tree of order 0 (or B_0 in short) is a single node tree
- The binomial tree of order 1 (B_1) is created from two B_0 trees, by making one B_0 tree the child of the other.
- The binomial tree of order 2 (B_2) is created from two B_1 trees, by making one B_1 tree the child of the other.
- and so on...

Example of a Binomial Heap – not examinable

A binomial heap containing $N = 19$ keys.

On the right is a binomial **heap** that contains a collection of binomial **trees**:

- **one** B_4 tree
- **zero** B_3 trees
- **zero** B_2 trees
- **one** B_1 tree
- **one** B_0 tree



Cool Properties

- The bit representation of N will immediately tell you which order trees are present in a Binomial Heap:

  $$N = (19)_{10} = (\overset{4}{1}\ \overset{3}{0}\ \overset{2}{0}\ \overset{1}{1}\ \overset{0}{1})_2$$

- In each Bionomial tree of degree/order d, if you count the number of nodes at any level, they are the Binomial coefficients of order d.
  - E.g. When $d = 4$, i.e., B_4 tree, you will see $\{1, 4, 6, 4, 1\}$ nodes in successive levels of that tree.

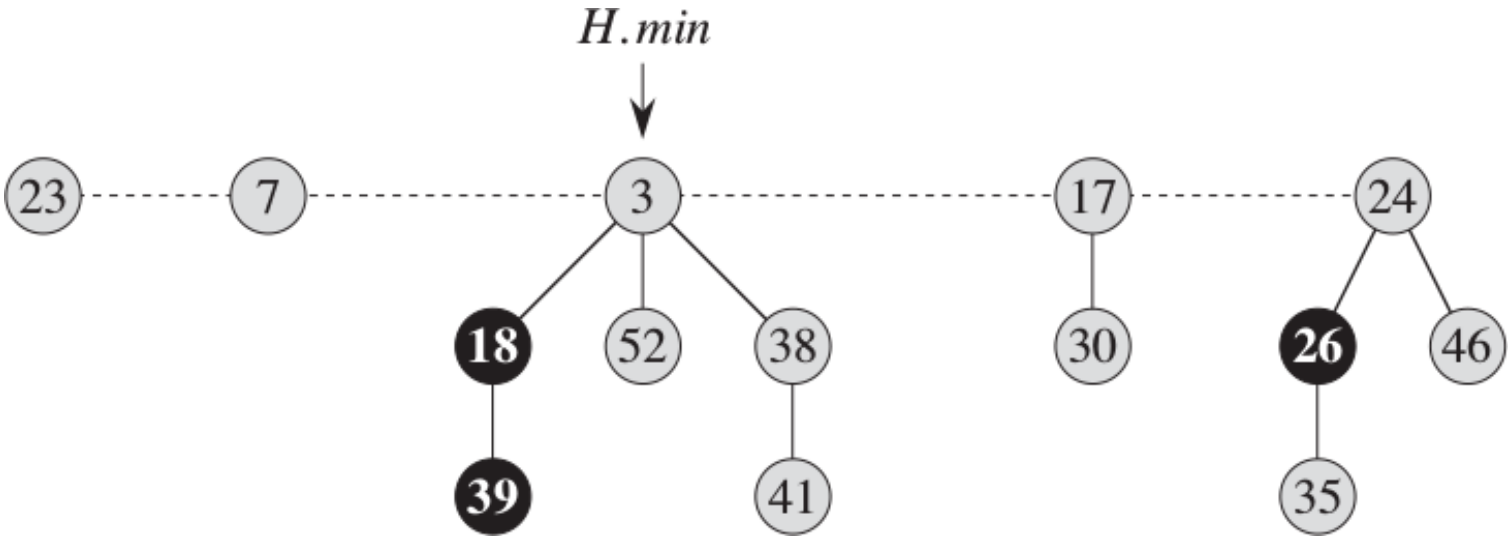
Fibonacci heaps vs Binomial heaps – not examinable

Similar to a **binomial heap**, a **Fibonacci heap** maintains a collection of (min-heap ordered) trees, however..

- ...the trees in the collection are **less strict** in their definitions, and..
  - ...while in a **binomial heap** 'consolidation' of trees is performed **eagerly** after each **extract-min** or **insert** operation...
  - ...in a **Fibonacci heap** the 'consolidation' is performed **lazily**, deferring it until **extract-min** operation is next performed.
  - We will see that the resulting tree structures in Fibonacci heaps are 'almost-but-not-quite-binomial' trees (in the number of nodes, i.e. size, of any tree in the heap).
    - ⋆ In binomial trees, $\text{size}(x) = 2^{\text{degree}(x)}$, where $\text{degree}(x)$ is the number of children of any node x.
    - ⋆ For trees in Fibonacci heap $\text{size}(x) \geq F_{\text{degree}(x)+2} \geq \phi^{\text{degree}(x)}$ (F_i denotes the ith Fibonacci number and $\phi = \frac{1+\sqrt{5}}{2} = 1.618033\ldots$ is the golden ratio.)

Example of a Fibonacci heap

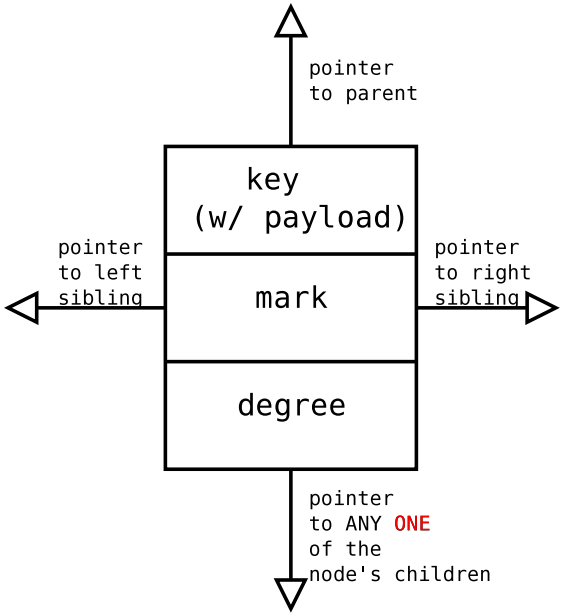
- A Fibonacci heap H containing 5 trees, with total 14 elements.



- $H.min$ is a pointer to root node (of a tree in the collection) with the minimum element.
- In a **Fibonacci heap**, each node/element is:
  - either **marked** (shown above as **black coloured nodes**)...
  - ...or **unmarked**/regular (shown as the grey coloured nodes above)
  - We will examine in later slides what this 'marking' means/does.

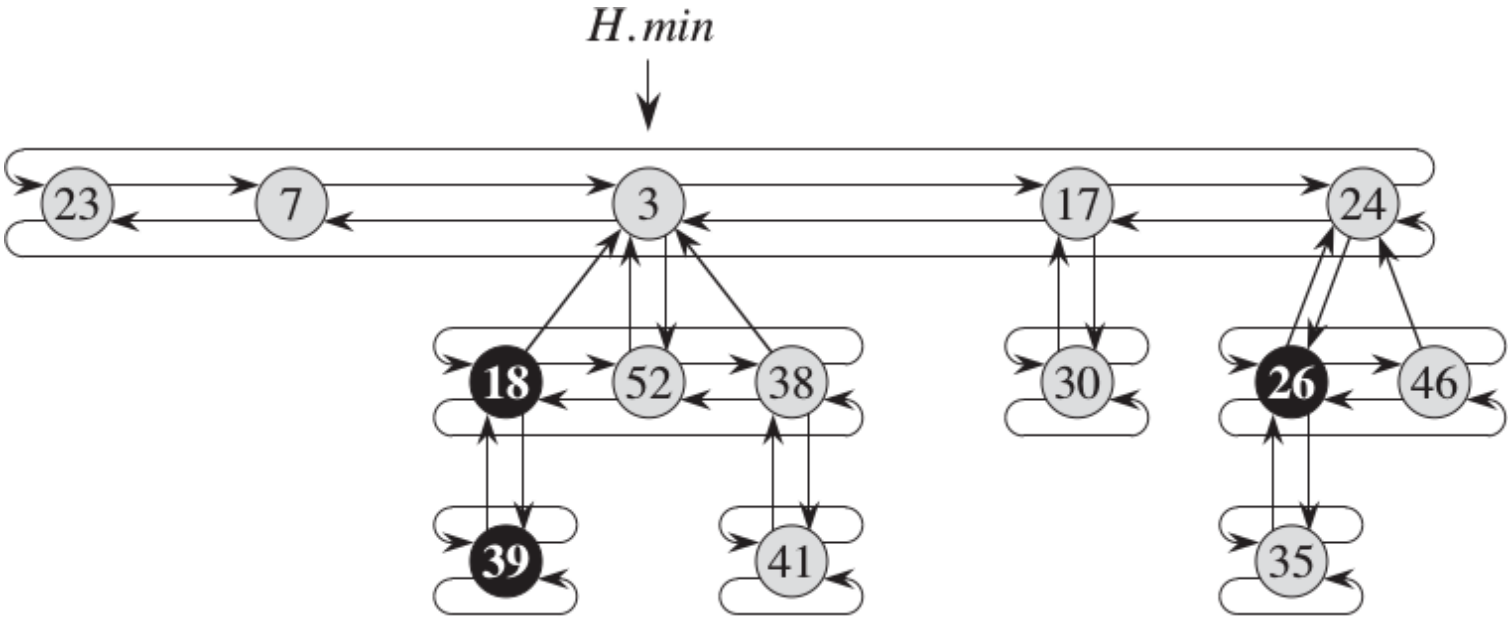
Representation of a node in a Fibonacci heap

- Each node x:
  - stores a **key**, and has associated **payload** information;
  - stores the number of its children: $x.degree$;
  - stores if the node is marked or not: $x.mark$;
  - has a pointer $x.parent$ to its parent node;
  - has a pointer $x.child$ to ANY ONE of its children.
  - x and its siblings form a **circular doubly linked list**:
    - ★ x has a pointer to its left sibling $x.left$...
    - ★ ...and to its right sibling $x.right$.



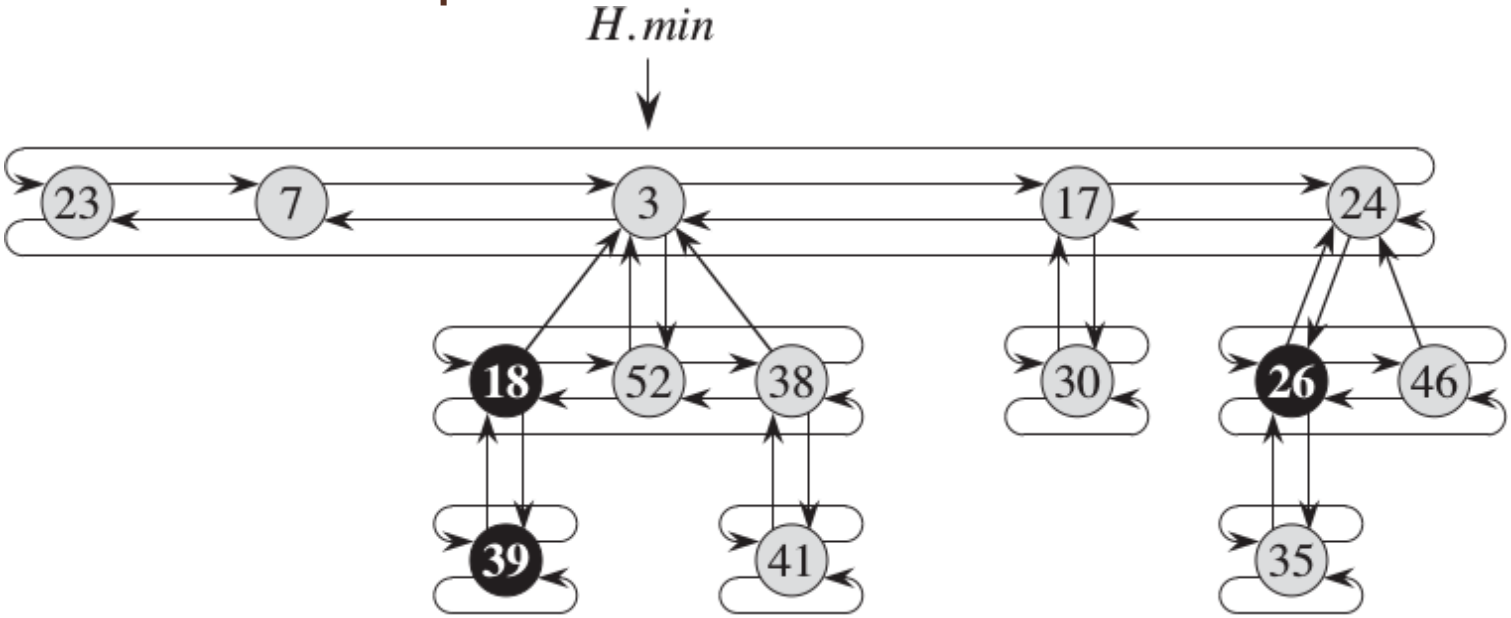
Fibonacci heaps are represented using circular doubly linked lists

- Below is a visualization of circular doubly linked list representation (and other pointers) for the example Fibonacci heap shown in slide # 14.



- This has several advantages:
  - One could **insert** an element into any location in $O(1)$ time.
  - One could **delete** an element from any location in $O(1)$ time.
  - One could **join** two lists of elements in $O(1)$ time.

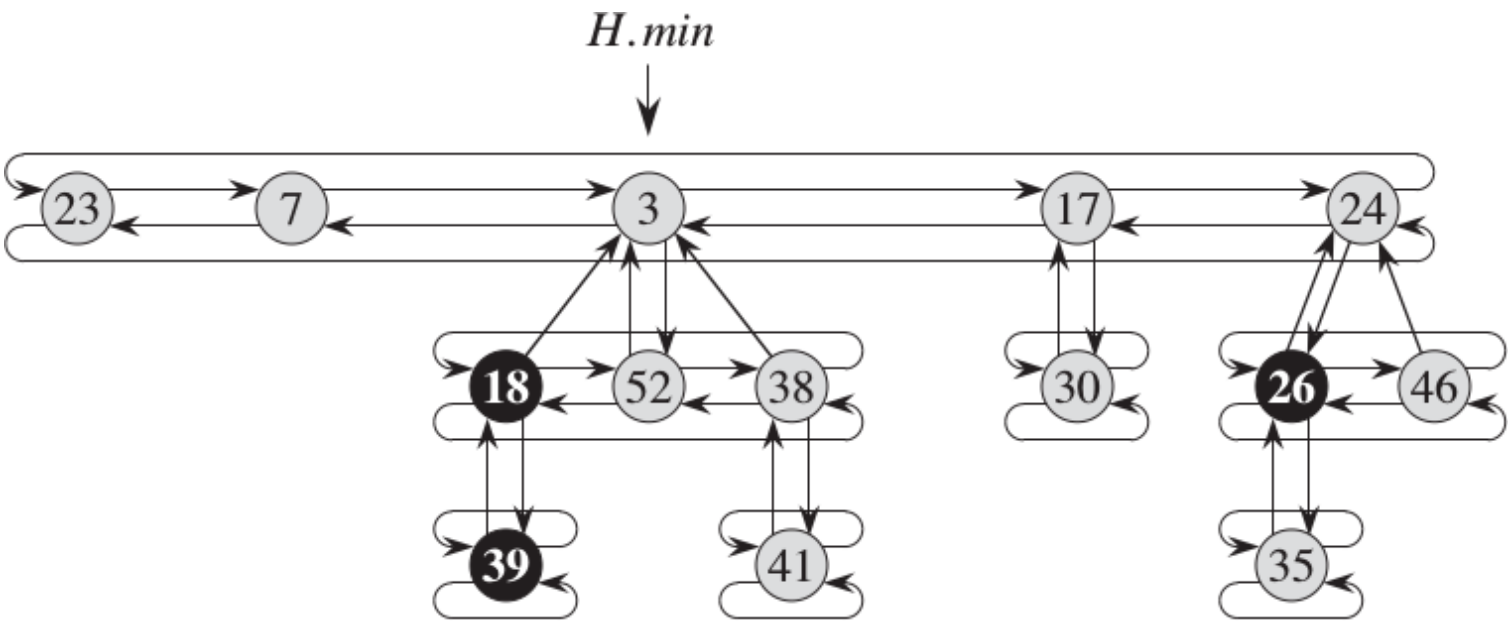
Fibonacci heaps – Attributes



Associated with each node/element x in a Fibonacci heap H, is:

- the **number of children** in the child list:
  - we will call the $degree$ of a node ($x.degree$).
  - Eg: (24) has $degree$=2. (7) has $degree$=0.
- whether a node is marked or not – $x.mark$
  - It will become clear in **decrease-key** operation what this means...
  - .. but quickly, '**marked**' implies the node has lost a child; **unmarked**' implies it hasn't lost a child. Details when slides #36-39 are covered.
  - Eg: (18) is '**marked**'. (30) is '**unmarked**'.

Fibonacci heaps – Attributes (continued)



- Access to the Fibonacci heap H is via the pointer to the **minimum** (key) node in the entire heap, denoted by $H.min$.
- Roots of all trees in the Fibonacci heap are connected by a **root_list**,
- ...where each tree's root can be accessed via $left$ and $right$ pointers, starting from $H.min$.

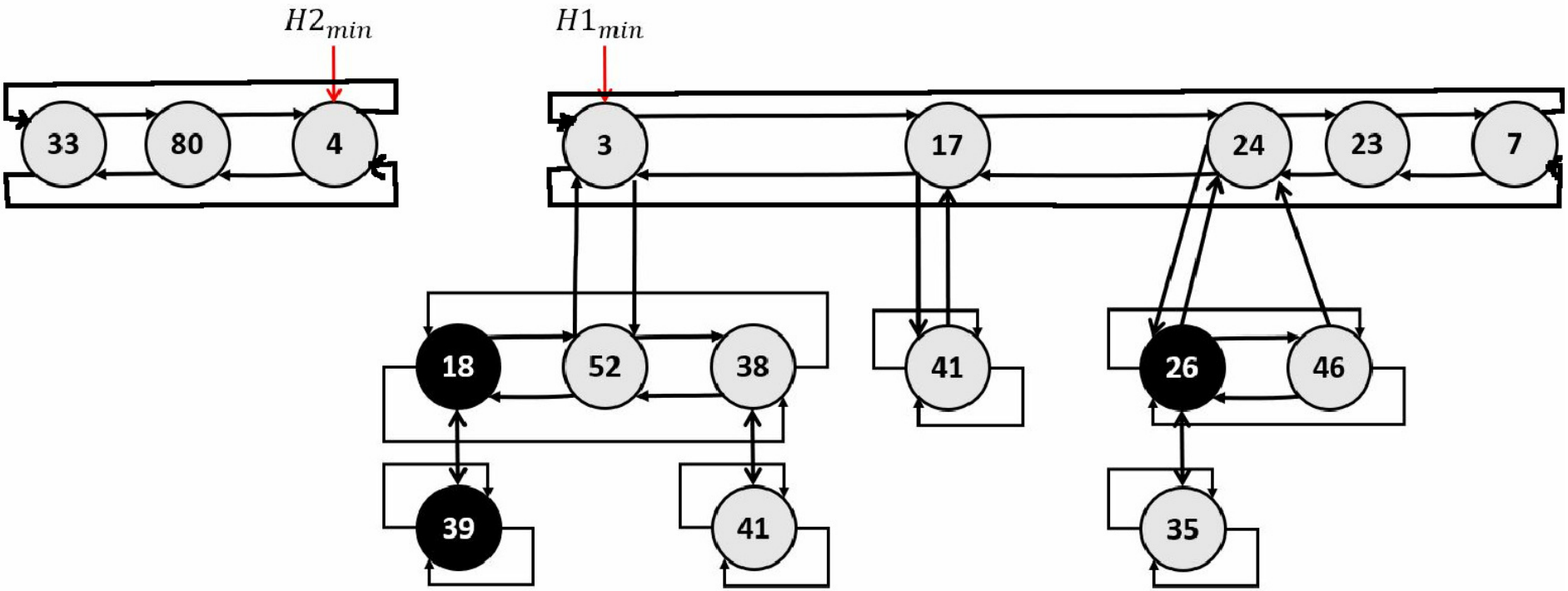
Fibonacci heaps – **merge** operation:

The **merge** operation involves combining two Fibonacci heaps, H_1 and H_2 into one:

- Takes $O(1)$ time. Why?
  - This involves combining two root lists...
  - ...each represented by a circular doubly-linked lists,
  - ... and accessible via their respective minimum (root) elements, $H_1.min$ and $H_2.min$...
  - ...before linking them into a single heap.

Fibonacci heaps – **merge** example:

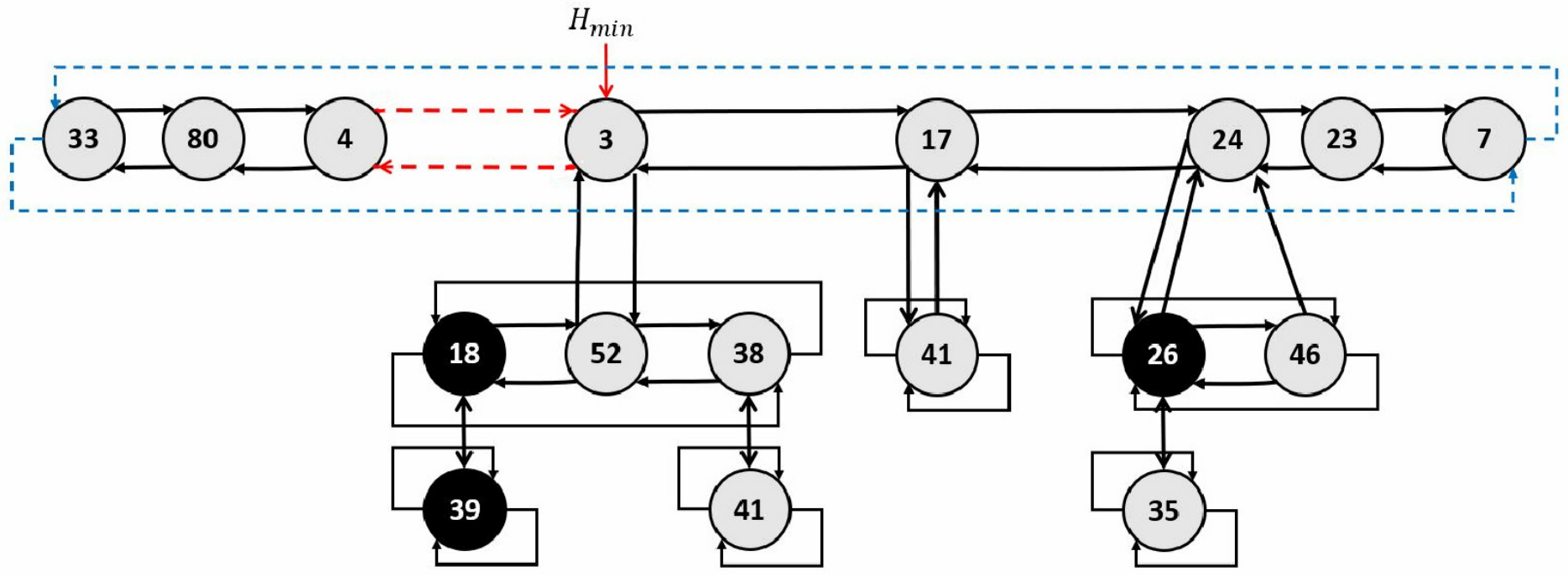
Before merge



- We can merge the two heaps by making the left sibling of $H1_{min}$ the left sibling of the right sibling of $H2_{min}$, i.e. point 33 to 7 and vice versa...
- ... and by making $H2_{min}$ the left sibling of $H1_{min}$, i.e. point 4 to 3 and vice versa.
- Set H_{min} to the minimum of $H1_{min}$ and $H2_{min}$.

Fibonacci heaps – **merge** example:

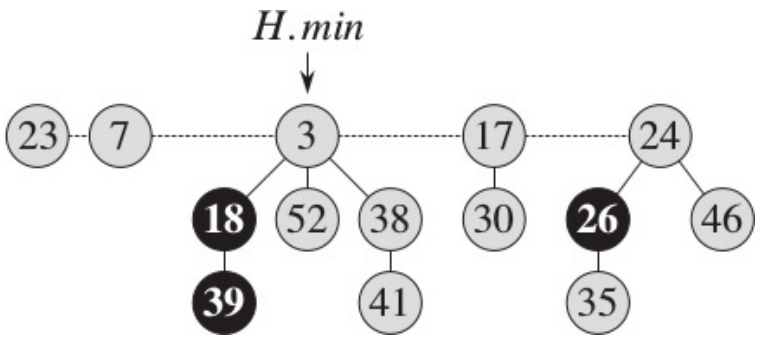
After merge



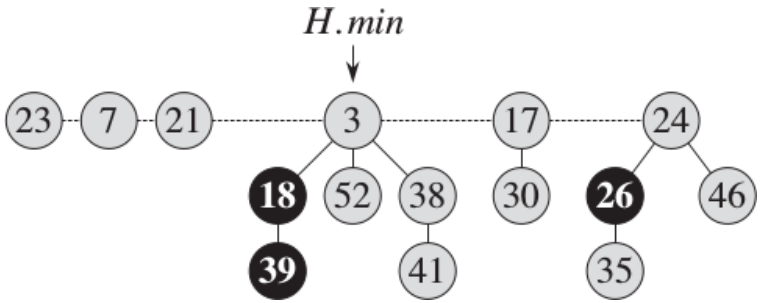
- We can merge the two heaps by making the left sibling of $H1_{min}$ the left sibling of the right sibling of $H2_{min}$, i.e. point (33) to (7) and vice versa...
- ... and by making $H2_{min}$ the left sibling of $H1_{min}$, i.e. point (4) to (3) and vice versa.
- Set H_{min} to the minimum of $H1_{min}$ and $H2_{min}$.

Fibonacci heaps – **insert** operation

insert(x) into a Fibonacci heap H. (In this example, $x = 21$.)



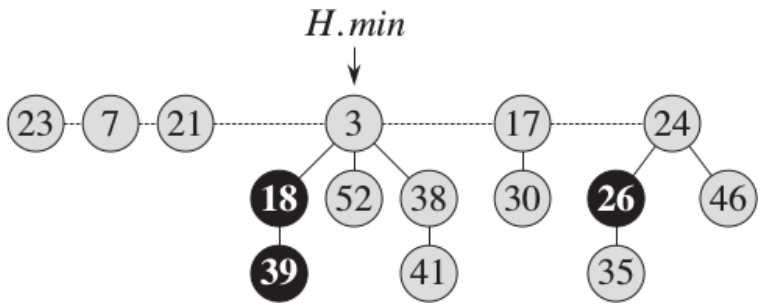
↓ **insert** 21



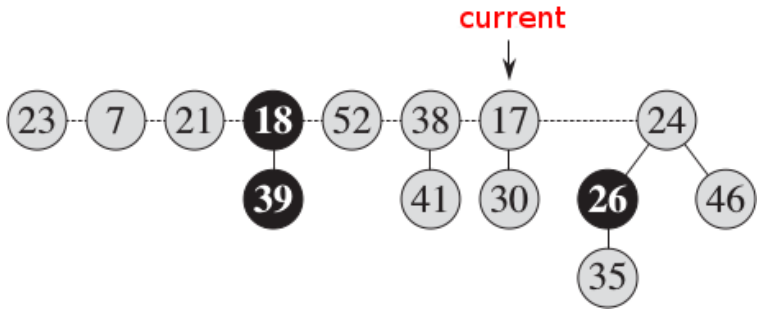
- Access H via the pointer $H.min$.
- **insert** x into the **root_list**, making it the left sibling of $H.min$ element/root.
- If $x < H.min$ (i.e., comparing their respective priorities/keys), update $H.min$ to point to x in the root level.
- Clearly, **insert**(x) is $O(1)$-time operation.

Fibonacci heaps – **extract-min** operation

Identify and delete the minimum (key) node in the heap



↓ **extract-min**



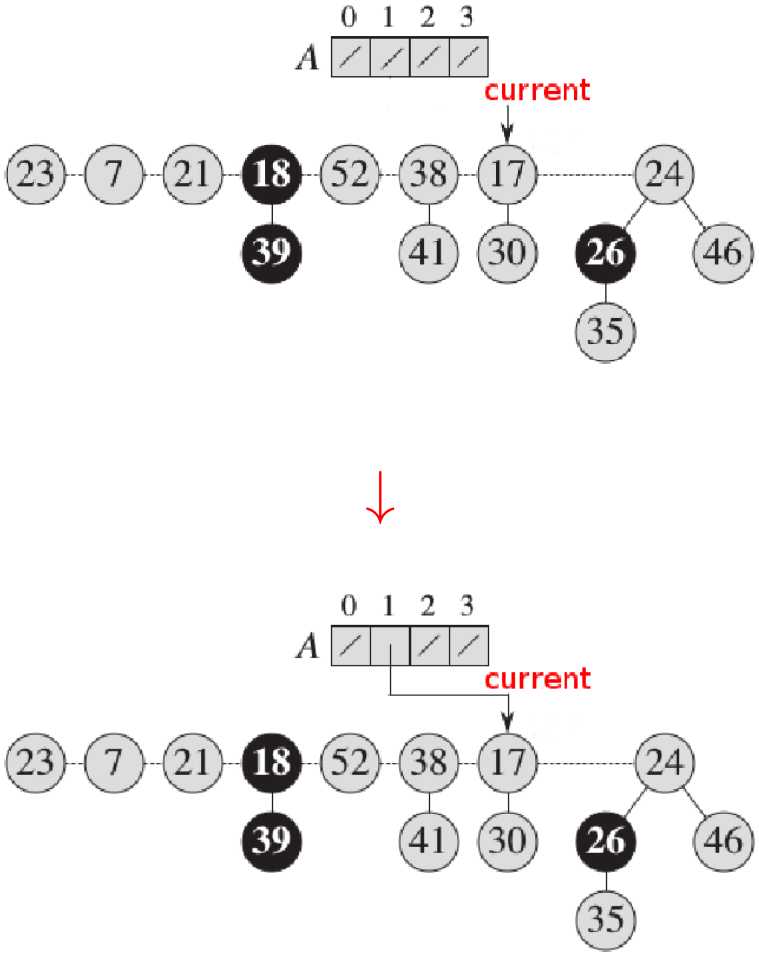
- Identify minimum element via the pointer $H.min$.
- To extract (and delete) minimum element (which is 3 in this running example),...
- ...set the current pointer to the $right$ sibling of $H.min$.
- ...promote/add all children (subtrees) to the root list, and
- IMPORTANT: Now run **consolidate** (or merge) operation. See next slides 23-28

NOTE!!!

consolidate operation ensures that no two nodes in the root level have the same degree (i.e., number of children).

Fibonacci heaps – **consolidate** operation

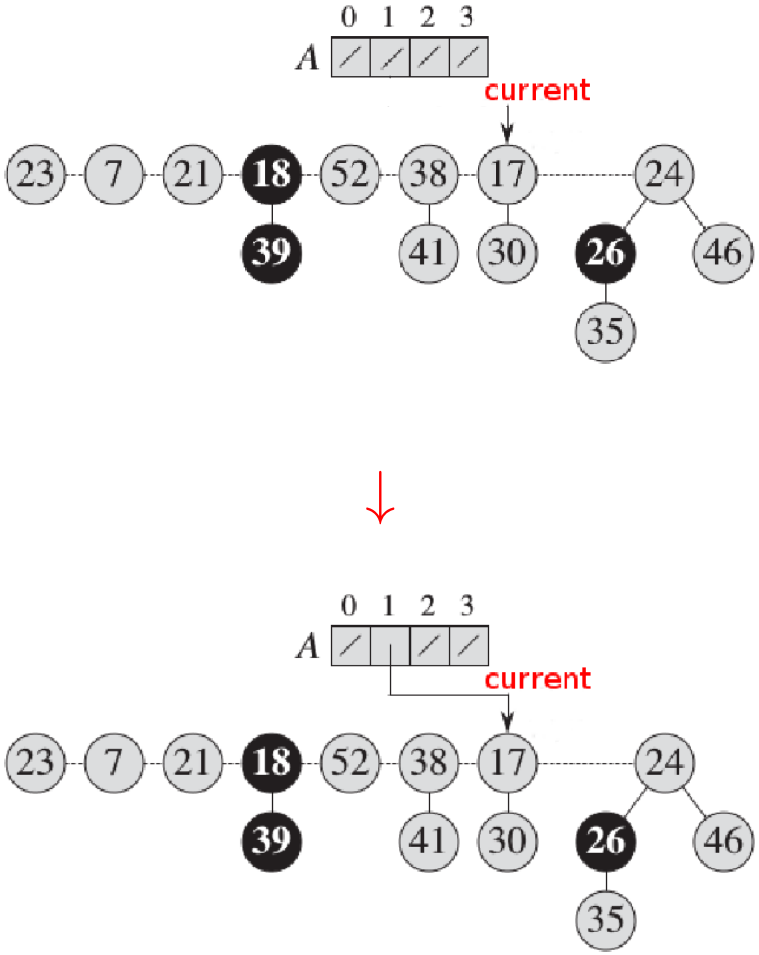
Fibonacci heaps run a **consolidate** operation only after a call to **extract-min**.



- Starting from current which is pointing to 17...
- Maintain an auxiliary array A to keep track of the root nodes indexed by their *degrees* (i.e., number of children). Initially, A is empty.
- Since the root node at current=17 has $degree = 1$...
- ...and $A[1]$ slot is empty, so...
- ...get $A[1]$ to point to the root node at current=17.
- Next, move current to the right sibling, i.e. current=24

Fibonacci heaps – **consolidate** operation

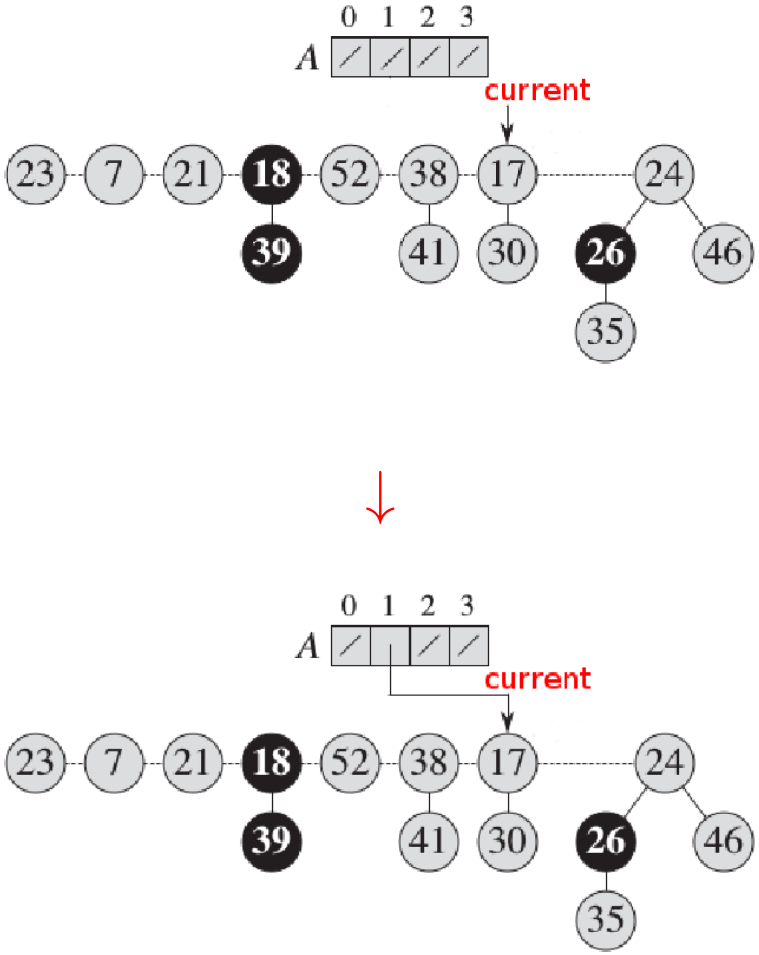
Fibonacci heaps run a **consolidate** operation only after a call to **extract-min**.



- Starting from current which is pointing to 17...
- Maintain an auxiliary array A to keep track of the root nodes indexed by their *degrees* (i.e., number of children). Initially, A is empty.
- Since the root node at current=17 has $degree = 1$...
- ...and $A[1]$ slot is empty, so...
- ...get $A[1]$ to point to the root node at current=17.
- Next, move current to the right sibling, i.e. current=24

Fibonacci heaps – **consolidate** operation

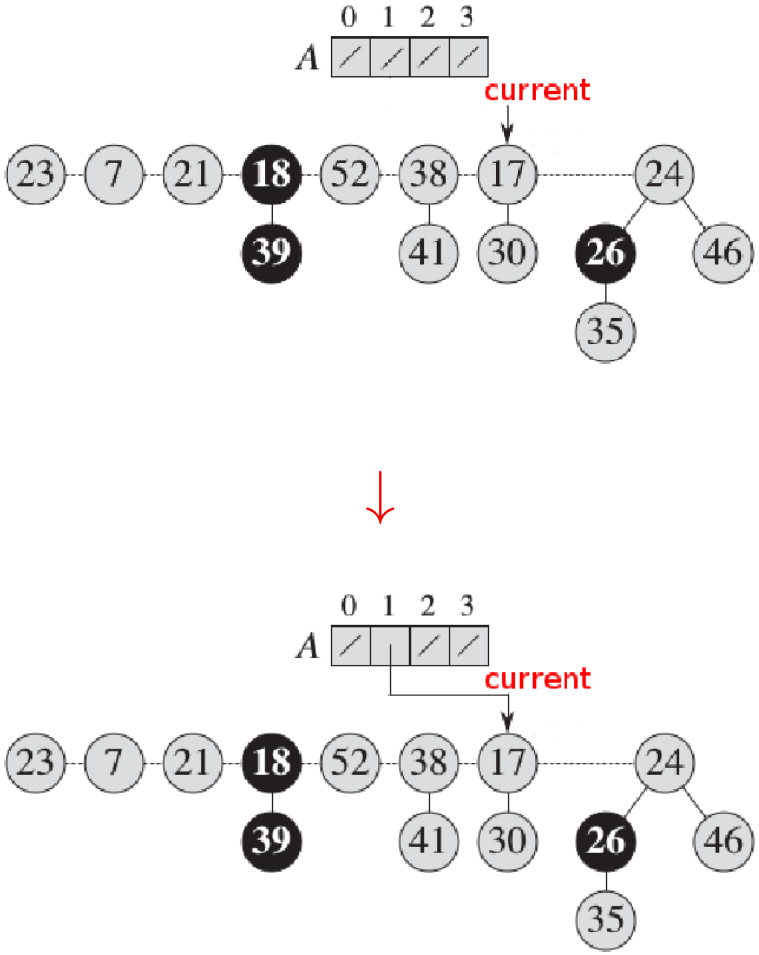
Fibonacci heaps run a **consolidate** operation only after a call to **extract-min**.



- Starting from current which is pointing to 17...
- Maintain an auxiliary array A to keep track of the root nodes indexed by their *degrees* (i.e., number of children). Initially, A is empty.
- Since the root node at current=17 has $degree = 1$...
- ...and $A[1]$ slot is empty, so...
- ...get $A[1]$ to point to the root node at current=17.
- Next, move current to the right sibling, i.e. current=24

Fibonacci heaps – **consolidate** operation

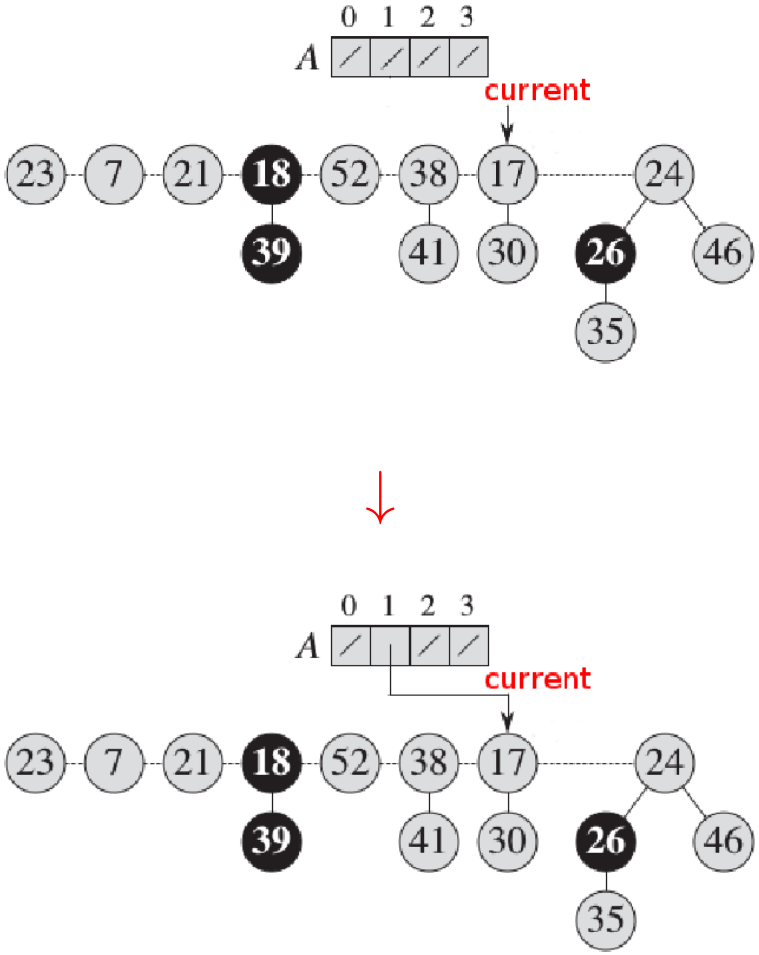
Fibonacci heaps run a **consolidate** operation only after a call to **extract-min**.



- Starting from current which is pointing to 17...
- Maintain an auxiliary array A to keep track of the root nodes indexed by their $degrees$ (i.e., number of children). Initially, A is empty.
- Since the root node at current=17 has $degree = 1$...
- ...and $A[1]$ slot is empty, so...
- ...get $A[1]$ to point to the root node at current=17.
- Next, move current to the right sibling, i.e. current=24

Fibonacci heaps – **consolidate** operation

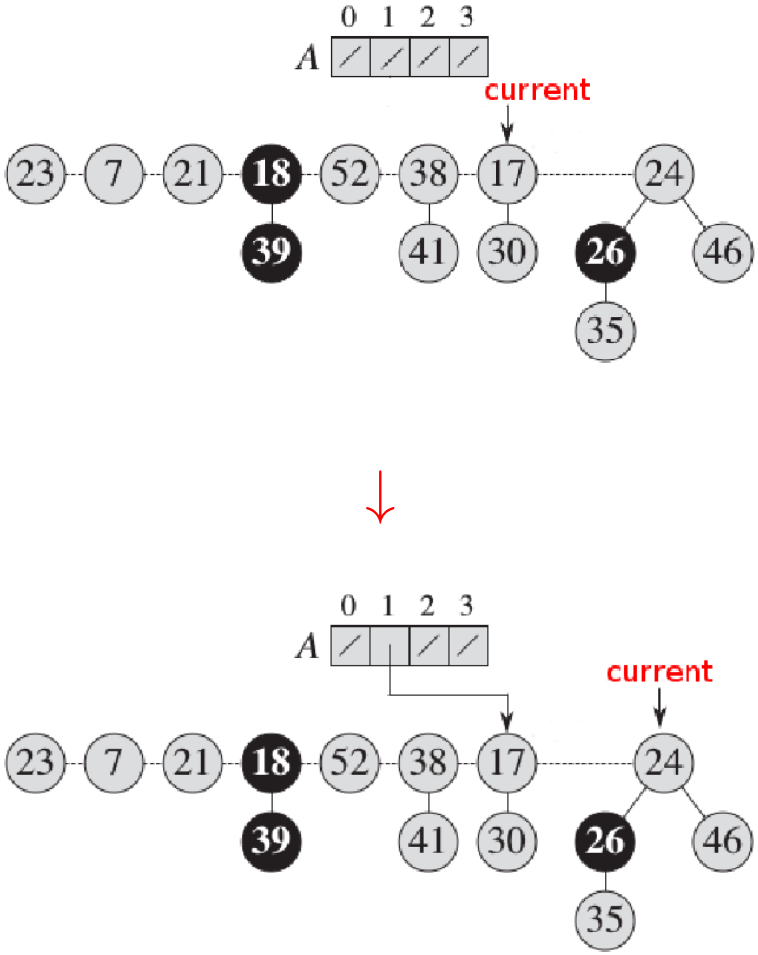
Fibonacci heaps run a **consolidate** operation only after a call to **extract-min**.



- Starting from current which is pointing to 17...
- Maintain an auxiliary array A to keep track of the root nodes indexed by their $degrees$ (i.e., number of children). Initially, A is empty.
- Since the root node at current=17 has $degree = 1$...
- ...and $A[1]$ slot is empty, so...
- ...get $A[1]$ to point to the root node at current=17.
- Next, move current to the right sibling, i.e. current=24

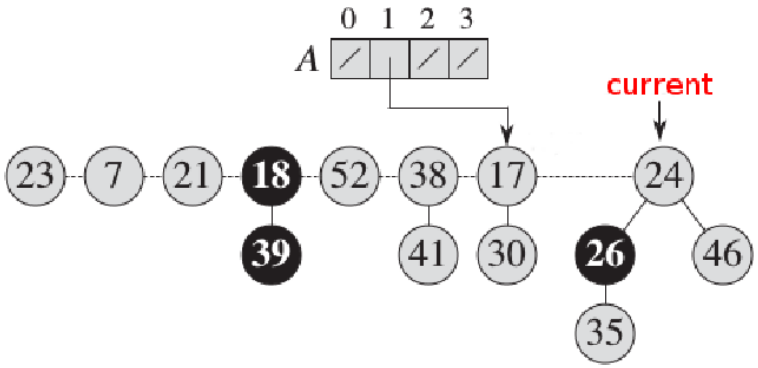
Fibonacci heaps – **consolidate** operation

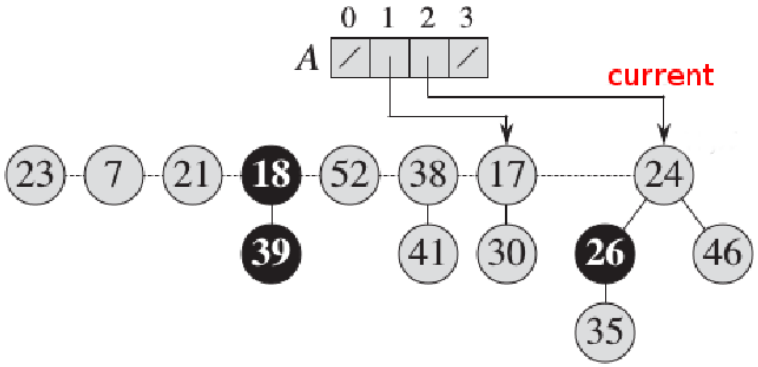
Fibonacci heaps run a **consolidate** operation only after a call to **extract-min**.



- Starting from current which is pointing to 17...
- Maintain an auxiliary array A to keep track of the root nodes indexed by their $degrees$ (i.e., number of children). Initially, A is empty.
- Since the root node at current=17 has $degree = 1$...
- ...and $A[1]$ slot is empty, so...
- ...get $A[1]$ to point to the root node at current=17.
- Next, move current to the right sibling, i.e. current=24

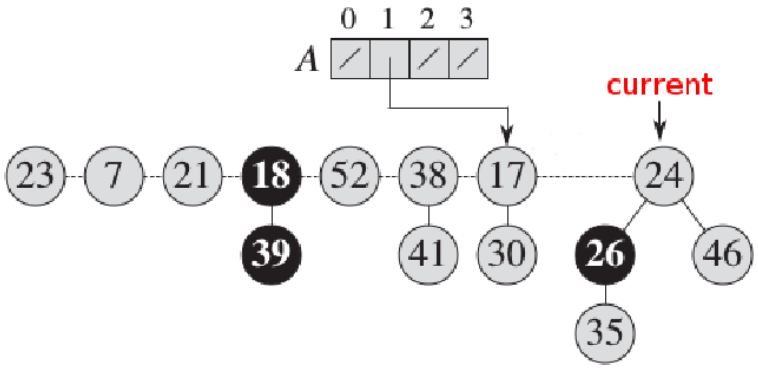
Fibonacci heaps – **consolidate** operation ...continued

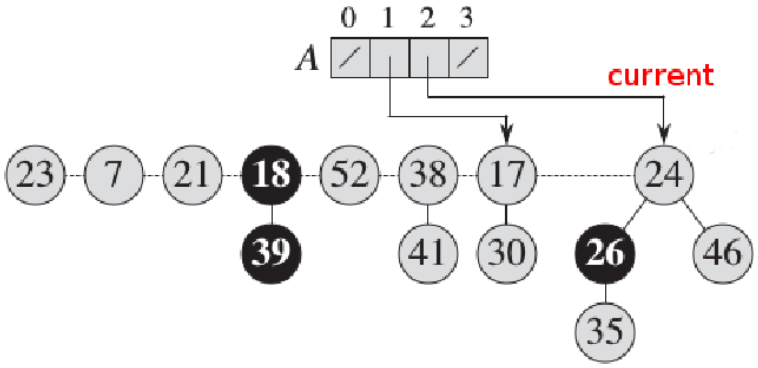




- Now, current=(24) has $degree$=2.
- Again, $A[2]$ is empty, so...
- ...get $A[2]$ to point to the root node at current =(24).
- Next, move current to the right sibling, i.e. move current from (24) to (23). Why?
  - `root_list` is a circular doubly linked list...
  - ...so the right sibling of (24) is (circularly) (23).
  - therefore, current=(23).

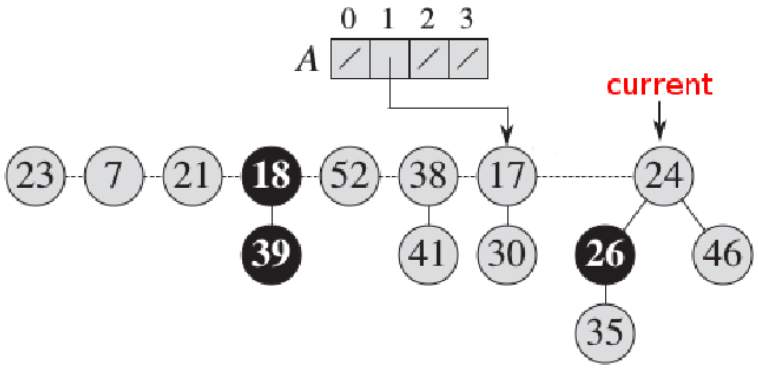
Fibonacci heaps – **consolidate** operation ...continued

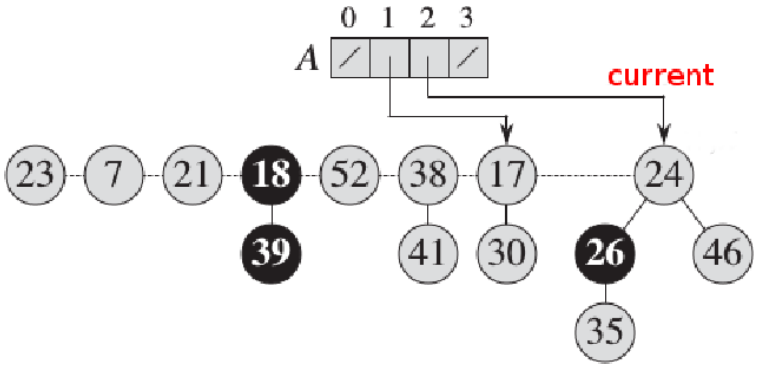




- Now, current=(24) has $degree$=2.
- Again, $A[2]$ is empty, so...
- ...get $A[2]$ to point to the root node at current =(24).
- Next, move current to the right sibling, i.e. move current from (24) to (23). Why?
  - `root_list` is a circular doubly linked list...
  - ...so the right sibling of (24) is (circularly) (23).
  - therefore, current=(23).

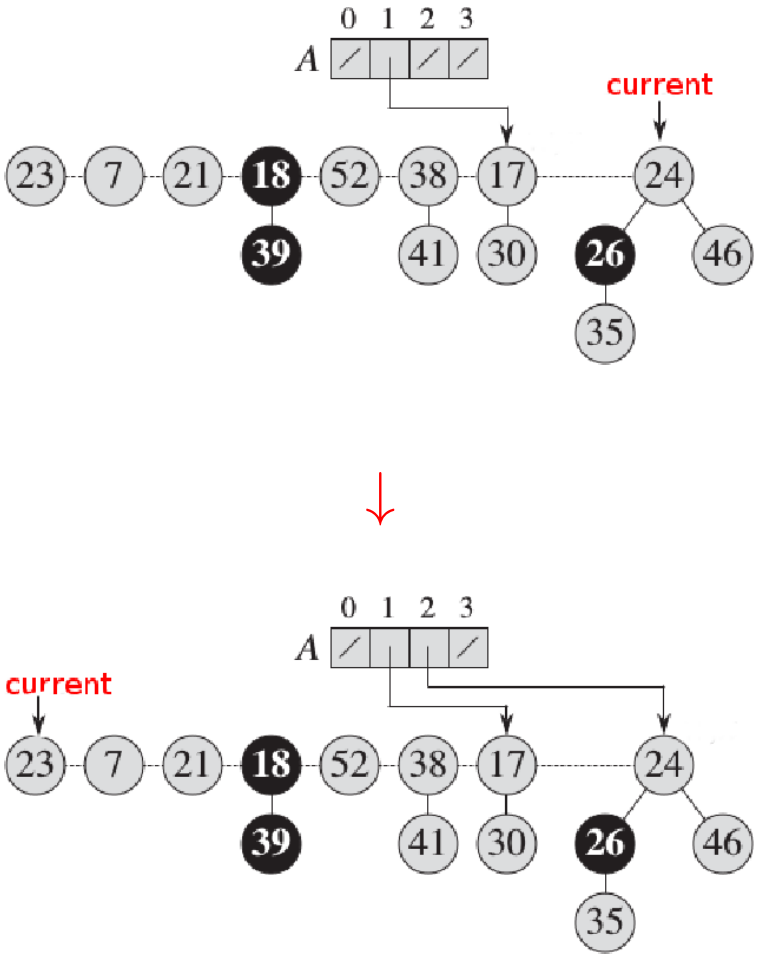
Fibonacci heaps – **consolidate** operation ...continued





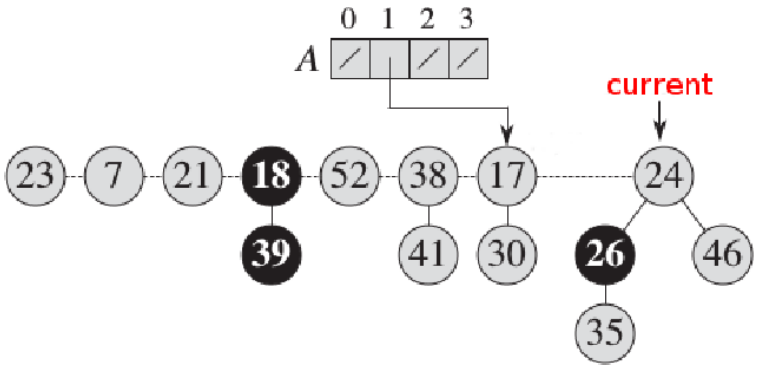
- Now, current=(24) has $degree$=2.
- Again, $A[2]$ is empty, so...
- ...get $A[2]$ to point to the root node at current =(24).
- Next, move current to the right sibling, i.e. move current from (24) to (23). Why?
  - `root_list` is a circular doubly linked list...
  - ...so the right sibling of (24) is (circularly) (23).
  - therefore, current=(23).

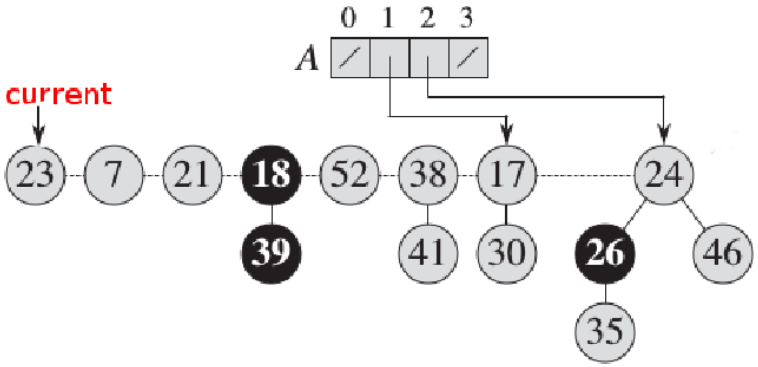
Fibonacci heaps – **consolidate** operation ...continued



- Now, current=(24) has $degree$=2.
- Again, $A[2]$ is empty, so...
- ...get $A[2]$ to point to the root node at current =(24).
- Next, move current to the right sibling, i.e. move current from (24) to (23). Why?
  - `root_list` is a circular doubly linked list...
  - ...so the right sibling of (24) is (circularly) (23).
  - therefore, current=(23).

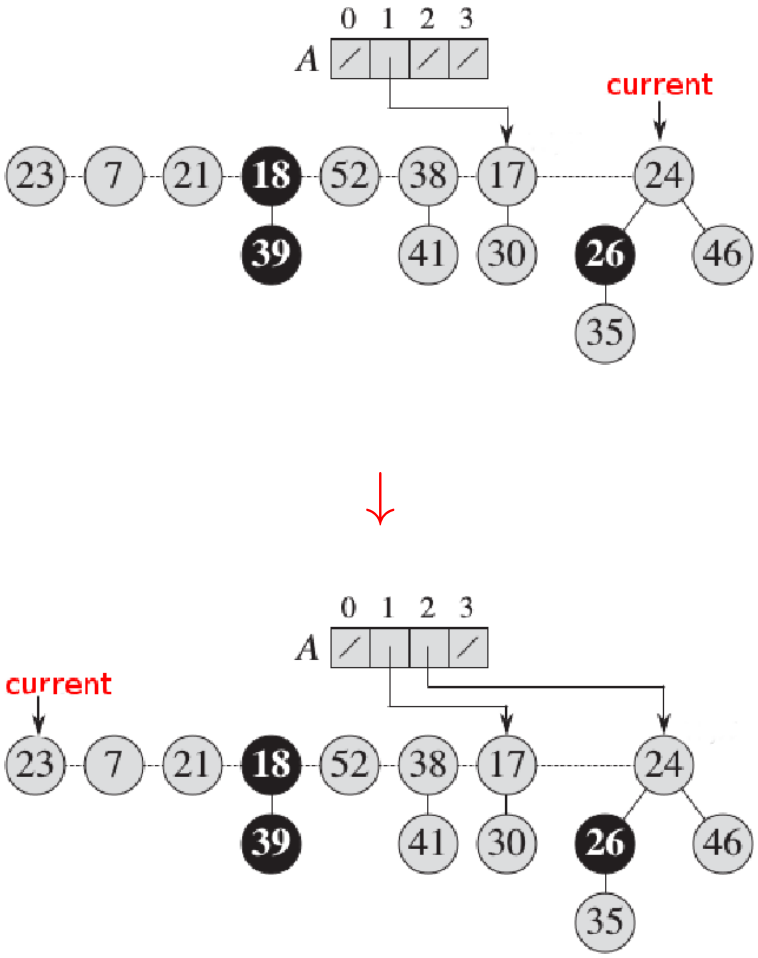
Fibonacci heaps – **consolidate** operation ...continued





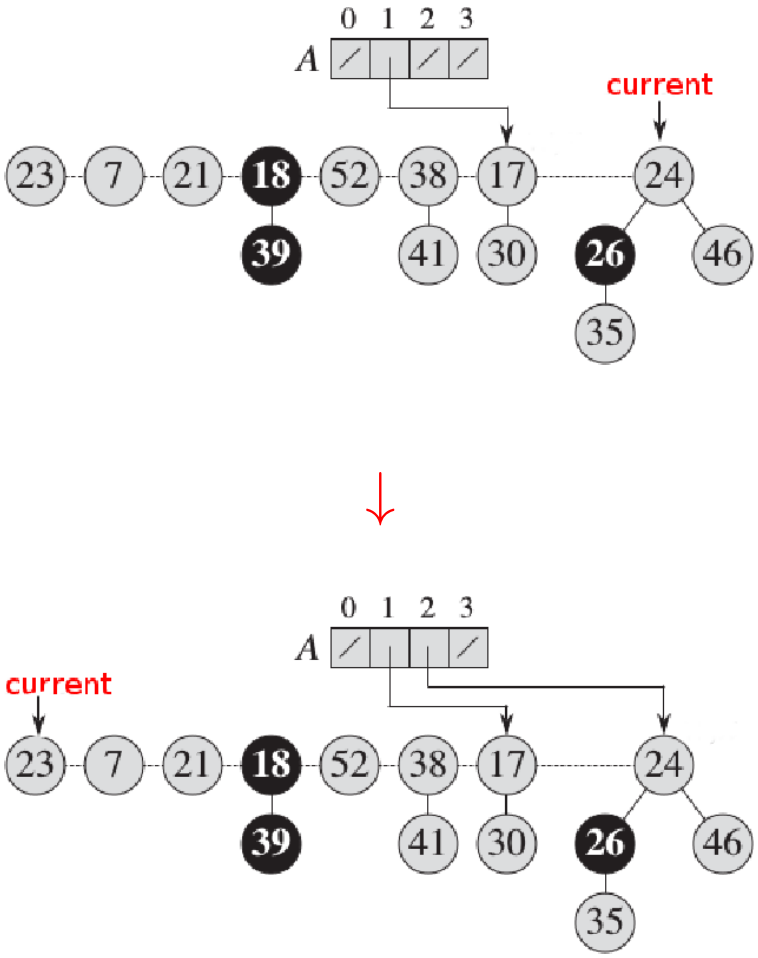
- Now, current=(24) has $degree$=2.
- Again, $A[2]$ is empty, so...
- ...get $A[2]$ to point to the root node at current =(24).
- Next, move current to the right sibling, i.e. move current from (24) to (23). Why?
  - **root_list** is a circular doubly linked list...
  - ...so the right sibling of (24) is (circularly) (23).
  - therefore, current=(23).

Fibonacci heaps – **consolidate** operation ...continued



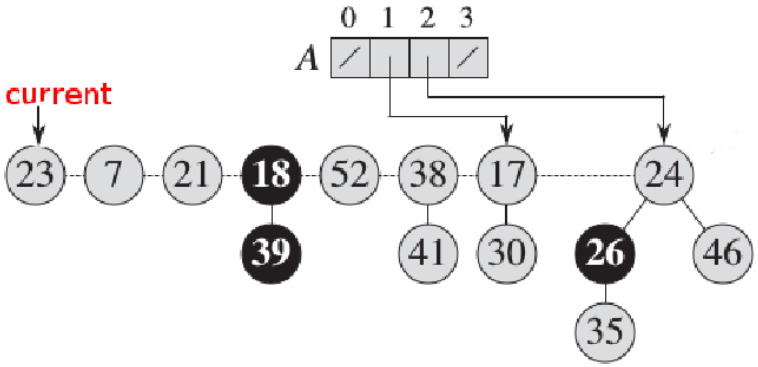
- Now, current=(24) has $degree$=2.
- Again, $A[2]$ is empty, so...
- ...get $A[2]$ to point to the root node at current =(24).
- Next, move current to the right sibling, i.e. move current from (24) to (23). Why?
  - **root_list** is a circular doubly linked list...
  - ...so the right sibling of (24) is (circularly) (23).
  - therefore, current=(23).

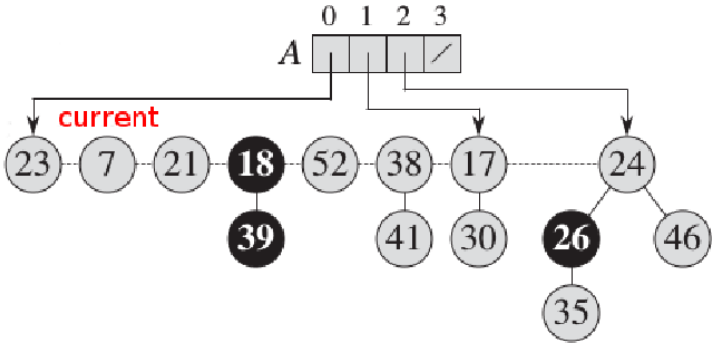
Fibonacci heaps – **consolidate** operation ...continued



- Now, current=(24) has $degree$=2.
- Again, $A[2]$ is empty, so...
- ...get $A[2]$ to point to the root node at current =(24).
- Next, move current to the right sibling, i.e. move current from (24) to (23). Why?
  - **root_list** is a circular doubly linked list...
  - ...so the right sibling of (24) is (circularly) (23).
  - therefore, current=(23).

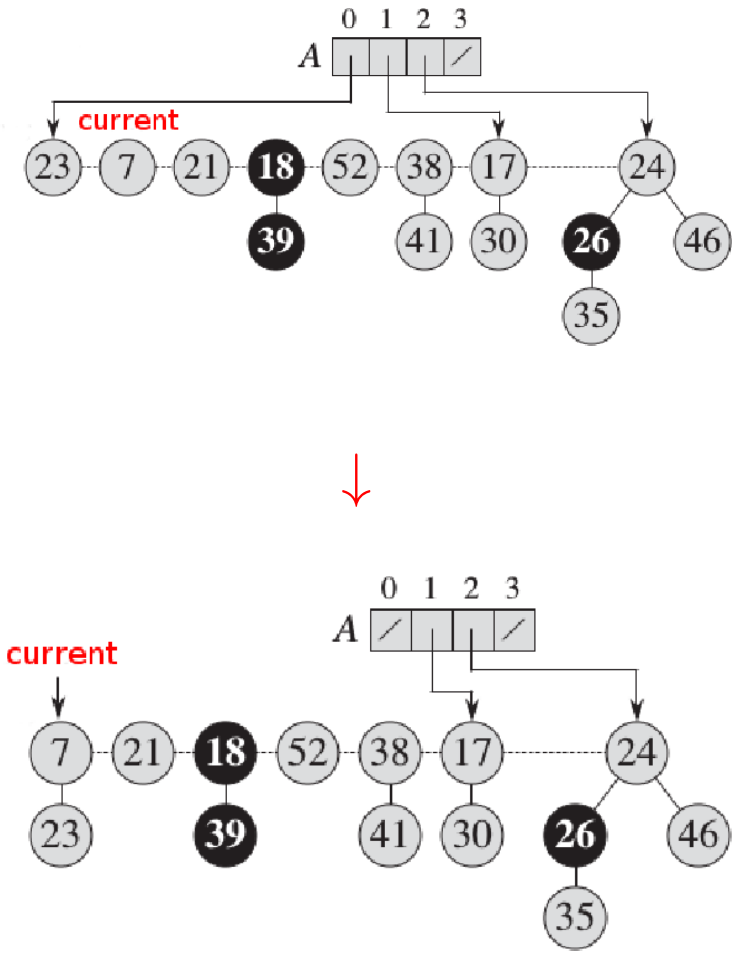
Fibonacci heaps – **consolidate** operation ...continued





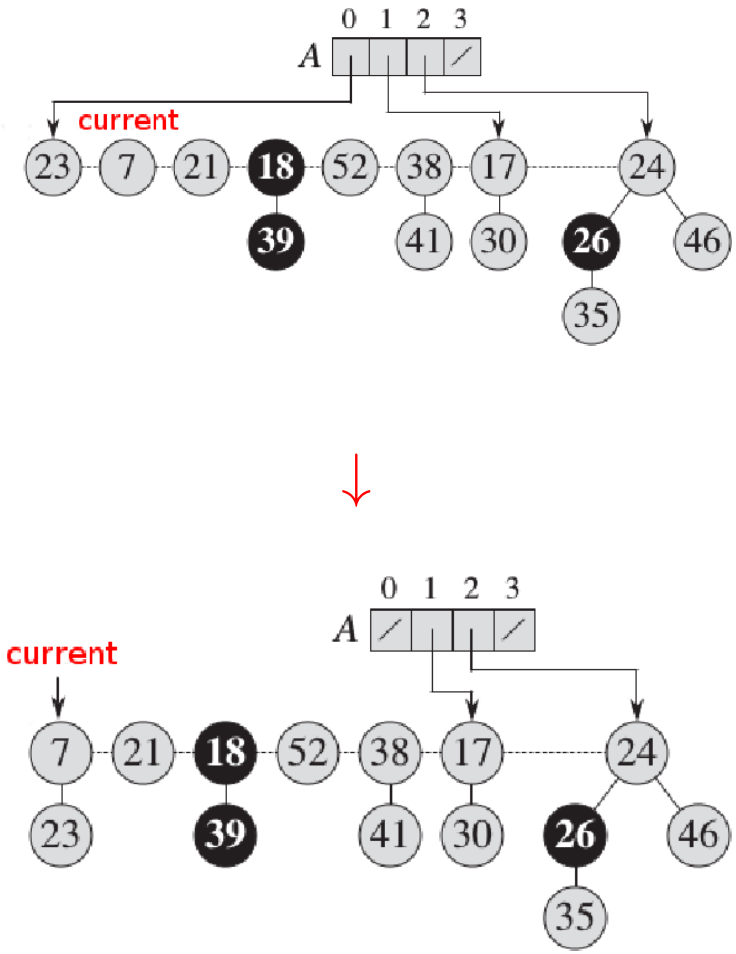
- Now, current=(23) has $degree$=0.
- $A[0]$ is empty, so...
- ...get $A[0]$ to point to the root node at current =(23).
- Next, move current to the right sibling, i.e. current=(7).

Fibonacci heaps – **consolidate** operation ...continued



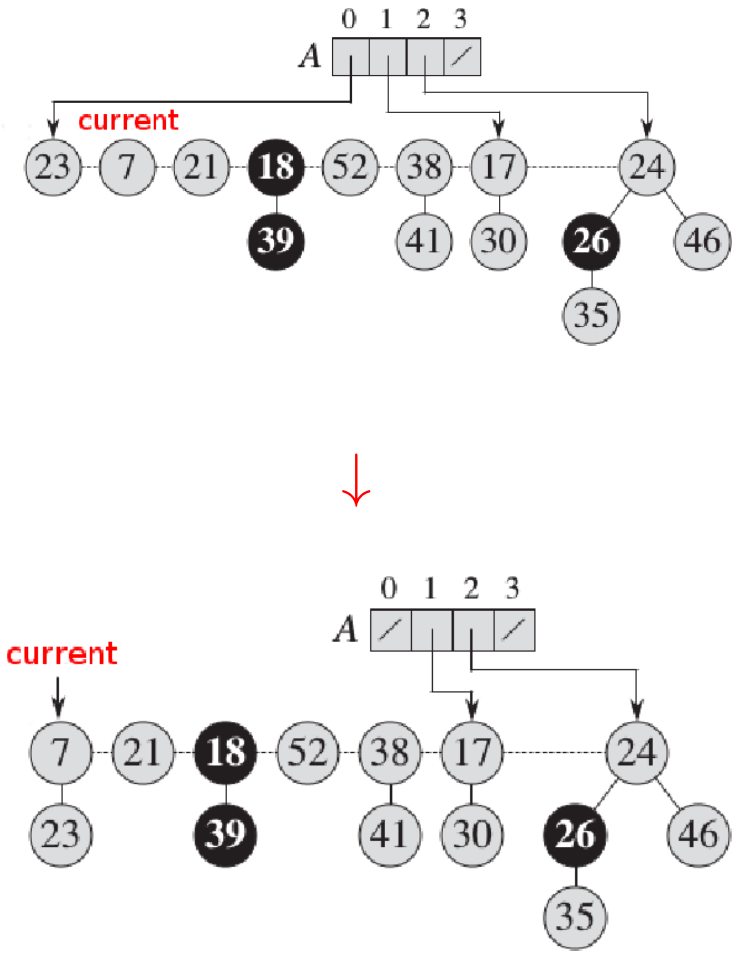
- Now, current=(7) has $degree$=0.
- But $A[0]$ is already **occupied** with a pointer to (23).
- Therefore, resolve this clash by **merging** (consolidating) trees with roots (7) and (23), and set $A[0]$ to empty.
- To maintain the (min-)heap property, root node (23) becomes the **child** of root node (7).
- current now points to the root of this merged tree, (7).
- Note: (7).$degree$ goes up from 0 to 1.

Fibonacci heaps – **consolidate** operation ...continued



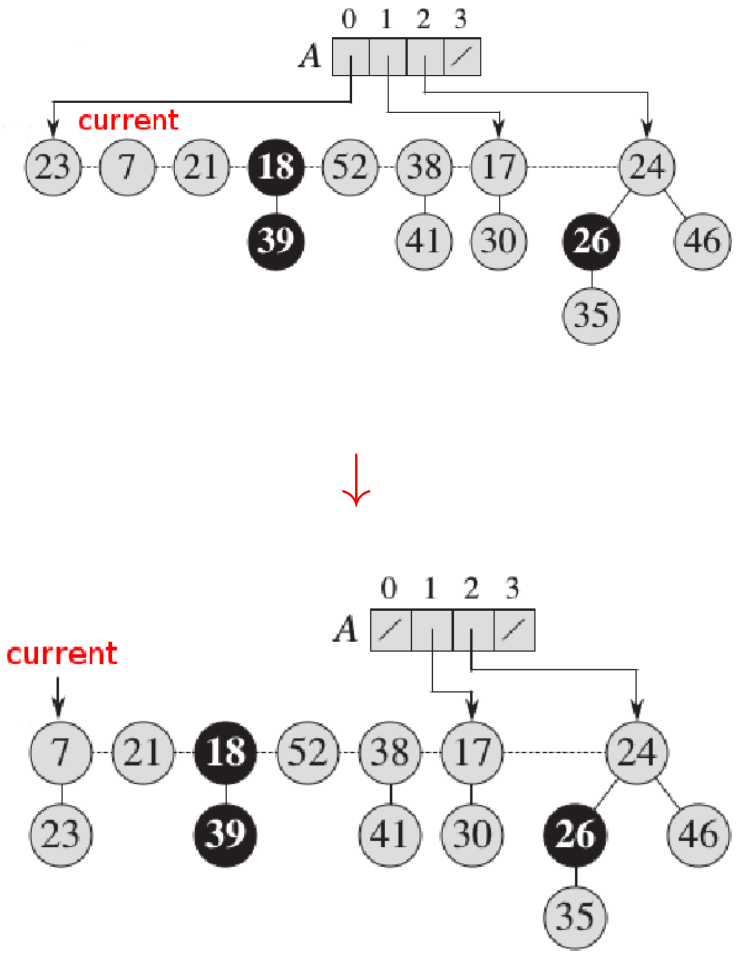
- Now, current=(7) has $degree$=0.
- But $A[0]$ is already **occupied** with a pointer to (23).
- Therefore, resolve this clash by **merging** (consolidating) trees with roots (7) and (23), and set $A[0]$ to empty.
- To maintain the (min-)heap property, root node (23) becomes the **child** of root node (7).
- current now points to the root of this merged tree, (7).
- Note: (7).$degree$ goes up from 0 to 1.

Fibonacci heaps – **consolidate** operation ...continued



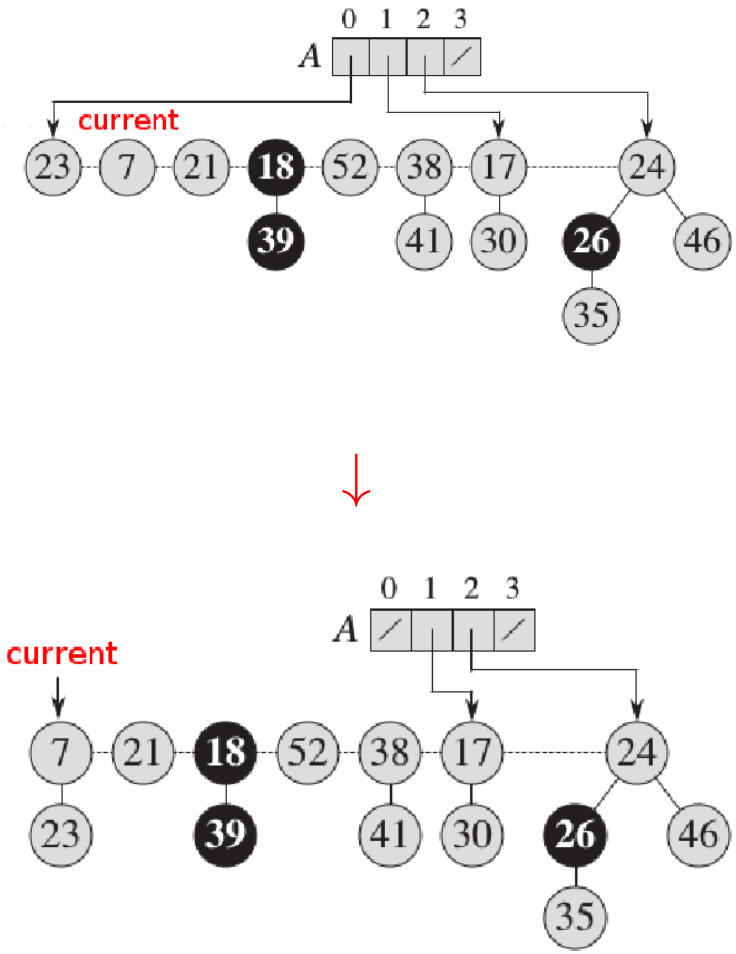
- Now, current=(7) has $degree$=0.
- But $A[0]$ is already **occupied** with a pointer to (23).
- Therefore, resolve this clash by **merging** (consolidating) trees with roots (7) and (23), and set $A[0]$ to empty.
- To maintain the (min-)heap property, root node (23) becomes the **child** of root node (7).
- current now points to the root of this merged tree, (7).
- Note: (7).$degree$ goes up from 0 to 1.

Fibonacci heaps – **consolidate** operation ...continued



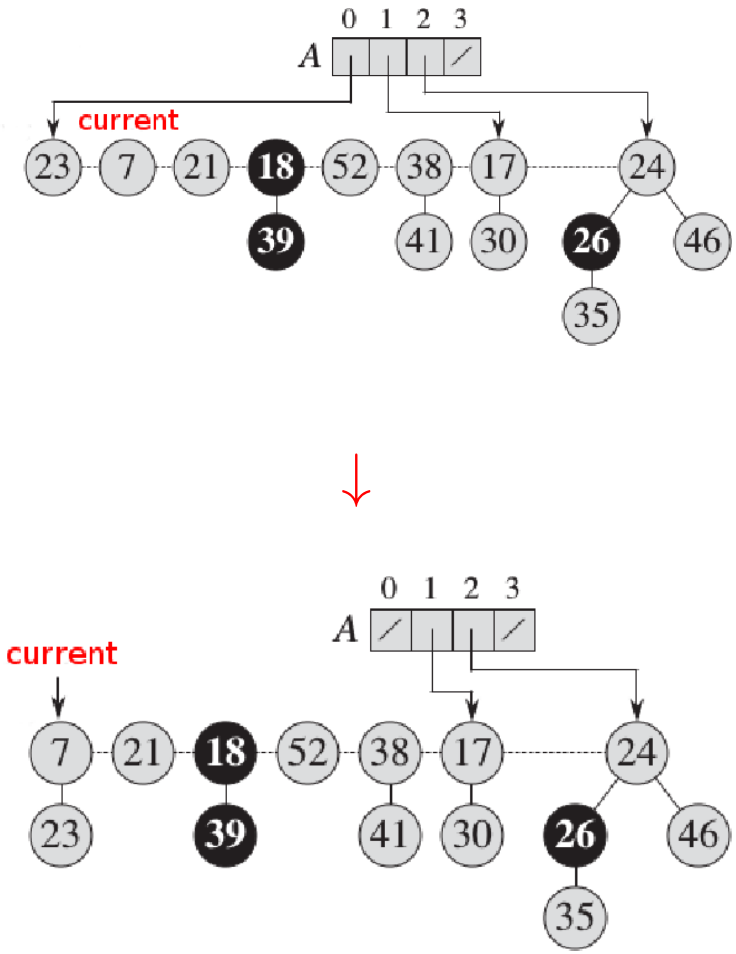
- Now, current=(7) has $degree$=0.
- But $A[0]$ is already **occupied** with a pointer to (23).
- Therefore, resolve this clash by **merging** (consolidating) trees with roots (7) and (23), and set $A[0]$ to empty.
- To maintain the (min-)heap property, root node (23) becomes the **child** of root node (7).
- current now points to the root of this merged tree, (7).
- Note: (7).$degree$ goes up from 0 to 1.

Fibonacci heaps – **consolidate** operation ...continued



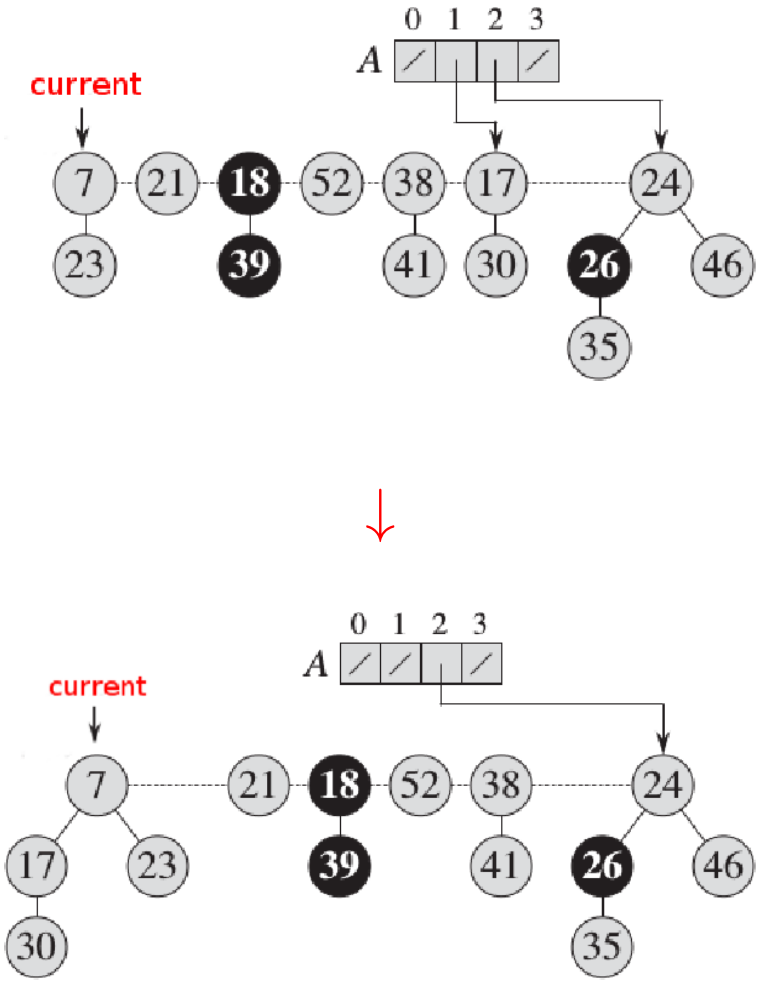
- Now, current=(7) has $degree$=0.
- But $A[0]$ is already **occupied** with a pointer to (23).
- Therefore, resolve this clash by **merging** (consolidating) trees with roots (7) and (23), and set $A[0]$ to empty.
- To maintain the (min-)heap property, root node (23) becomes the **child** of root node (7).
- current now points to the root of this merged tree, (7).
- Note: (7).$degree$ goes up from 0 to 1.

Fibonacci heaps – **consolidate** operation ...continued



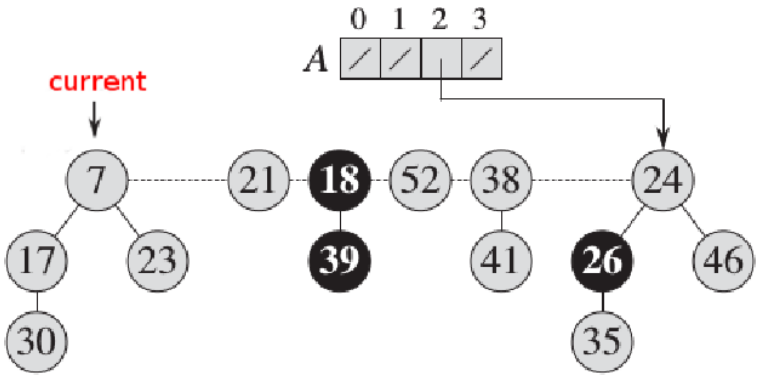
- Now, current=(7) has $degree$=0.
- But $A[0]$ is already **occupied** with a pointer to (23).
- Therefore, resolve this clash by **merging** (consolidating) trees with roots (7) and (23), and set $A[0]$ to empty.
- To maintain the (min-)heap property, root node (23) becomes the **child** of root node (7).
- current now points to the root of this merged tree, (7).
- Note: (7).$degree$ goes up from 0 to 1.

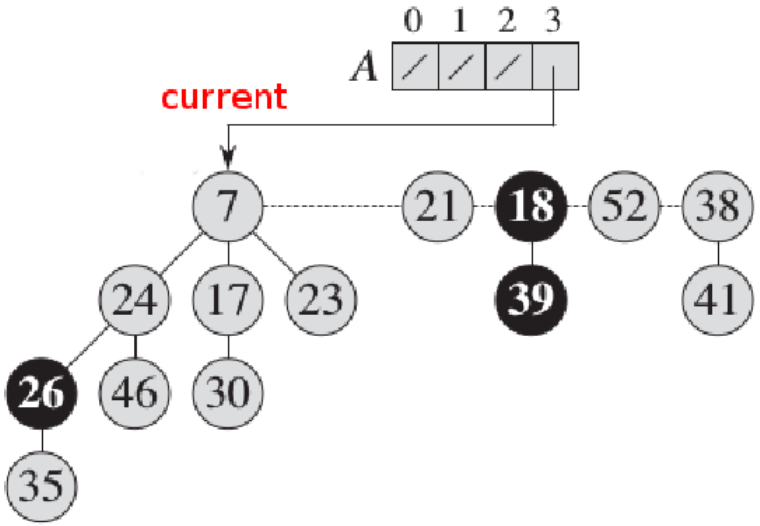
Fibonacci heaps – **consolidate** operation ...continued



- Repeat: current=(7) has $degree$=1.
- But $A[1]$ is already **occupied** with a pointer to (17).
- Therefore, resolve this clash by **merging** (consolidating) trees with roots (7) and (17), and set $A[1]$ to empty.
- To maintain the (min-)heap property, root node (17) becomes the **child** of root node (7).
- current now points to the root of this merged tree, (7).
- Note: (7).$degree$ goes up from 1 to 2.

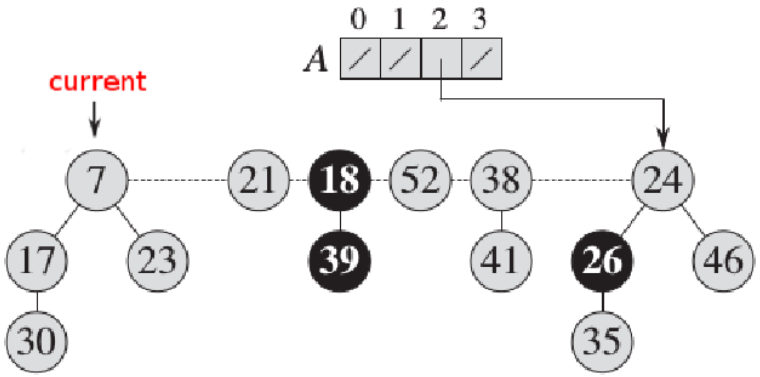
Fibonacci heaps – **consolidate** operation ...continued

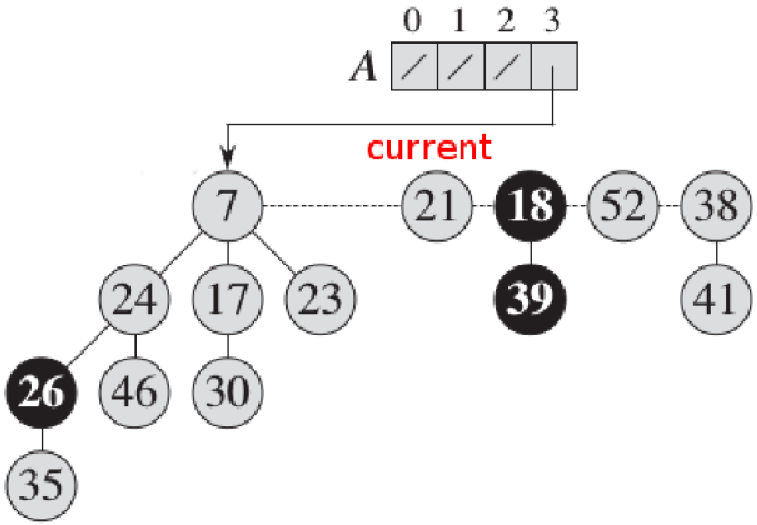




- Repeat: current=(7) has $degree$=2.
- But $A[2]$ is already **occupied** with a pointer to (24).
- Therefore, resolve this clash by **merging** (consolidating) trees with roots (7) and (24), and set $A[2]$ to empty.
- To maintain the (min-)heap property, root node (24) becomes the **child** of root node (7).
- current now points to the root of this merged tree, (7).
- Note: (7).$degree$ goes up from 2 to 3.
- Since, $A[3]$ is empty, get $A[3]$ to point to the root node at current=(7).
- Next, move current to the right sibling, i.e. current=(21)

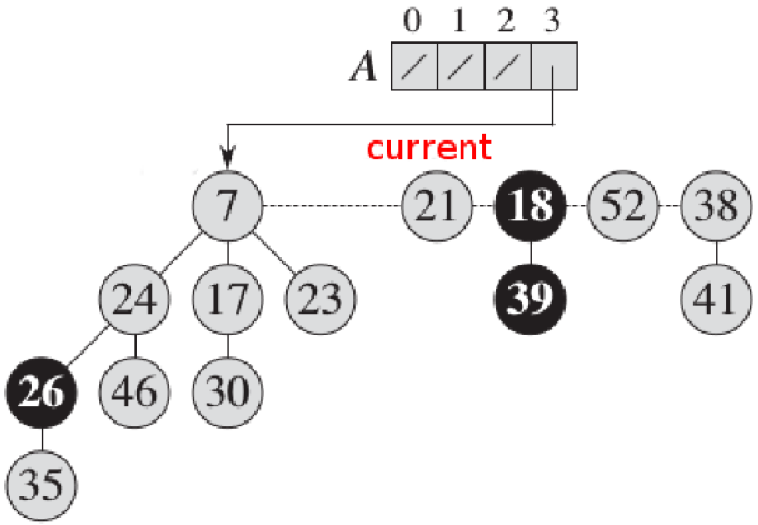
Fibonacci heaps – **consolidate** operation ...continued

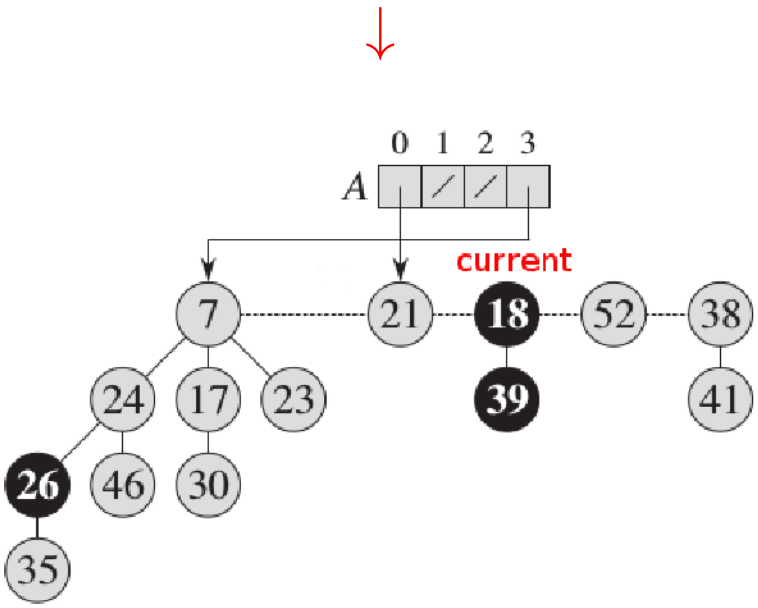




- Repeat: current=(7) has $degree$=2.
- But $A[2]$ is already **occupied** with a pointer to (24).
- Therefore, resolve this clash by **merging** (consolidating) trees with roots (7) and (24), and set $A[2]$ to empty.
- To maintain the (min-)heap property, root node (24) becomes the **child** of root node (7).
- current now points to the root of this merged tree, (7).
- Note: (7).$degree$ goes up from 2 to 3.
- Since, $A[3]$ is empty, get $A[3]$ to point to the root node at current=(7).
- Next, move current to the right sibling, i.e. current=(21)

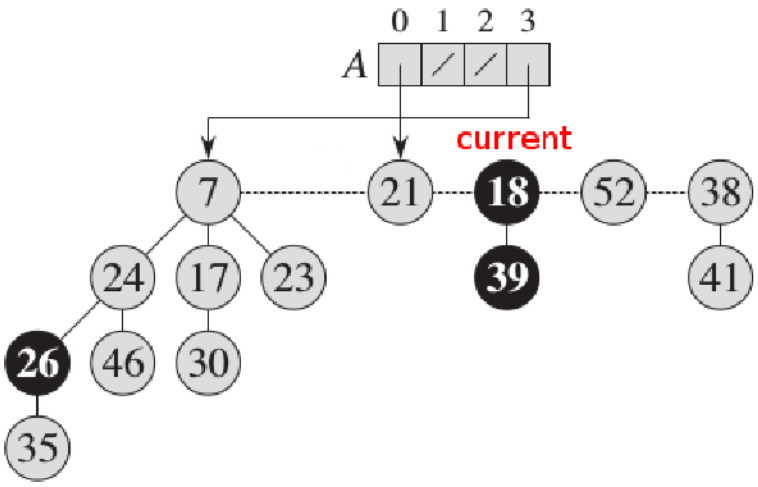
Fibonacci heaps – **consolidate** operation ...continued

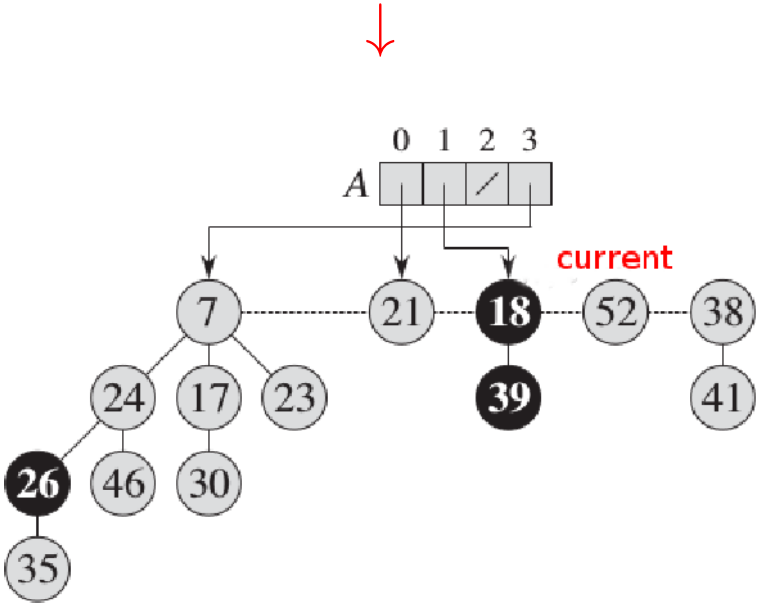




- Now, current=(21) has $degree$=0.
- $A[0]$ is empty, so...
- ...get $A[0]$ to point to the root node at current=(21).
- Next, move current to the right sibling, i.e current=(18).

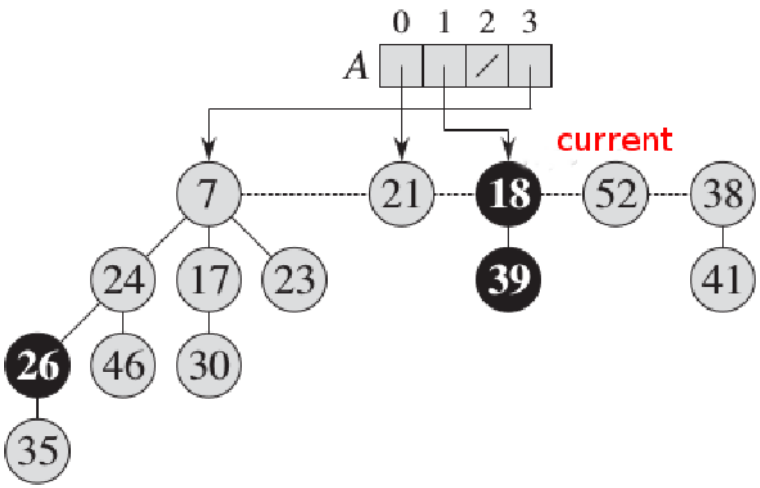
Fibonacci heaps – **consolidate** operation ...continued

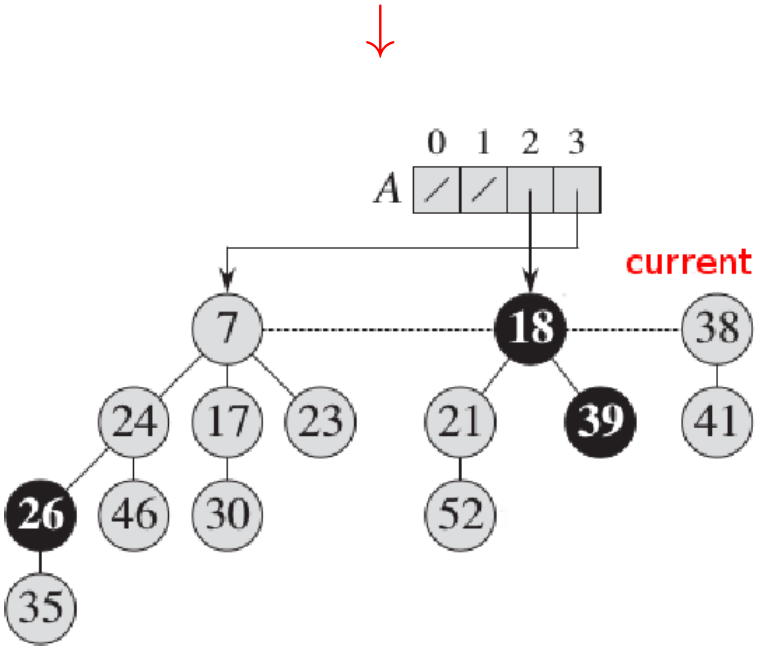




- Now, current=18 has $degree$=1.
- $A[1]$ is empty, so...
- ...get $A[1]$ to point to the root node at current=18.
- Next, move current to the right sibling, i.e current=52.

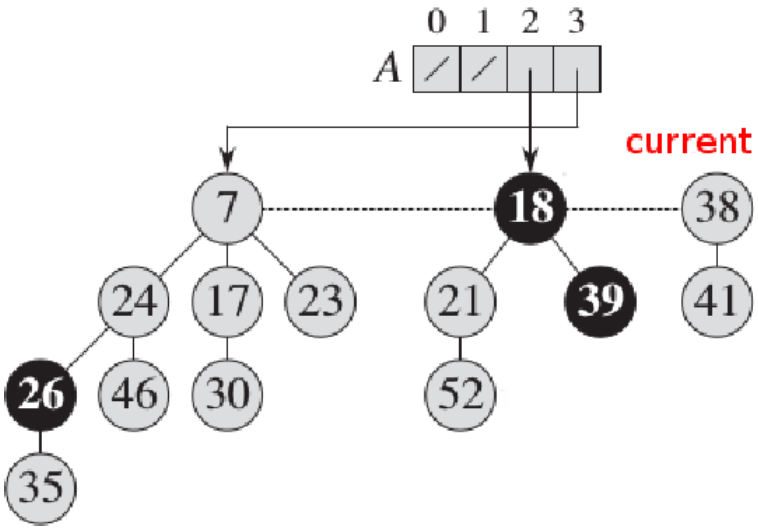
Fibonacci heaps – **consolidate** operation ...continued

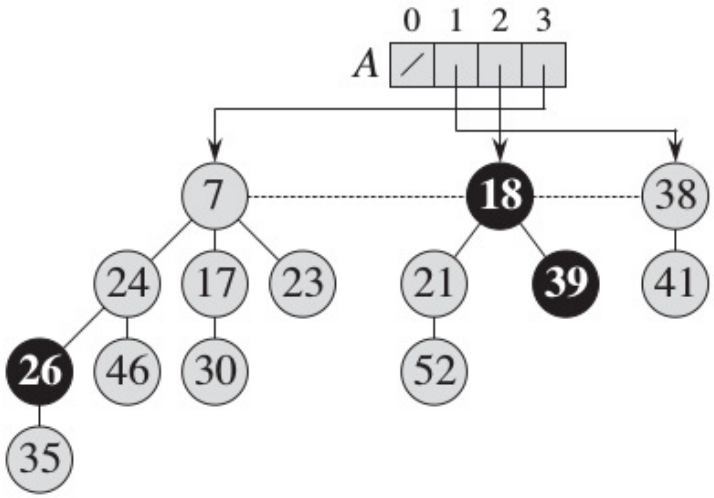




- Now, current=(52) has $degree$=0, but $A[0]$ is occupied.
- So, in operations similar to those on slides #26-28...
- ...we get to the state shown in the figure on the left (below).
- current now points to root (38).

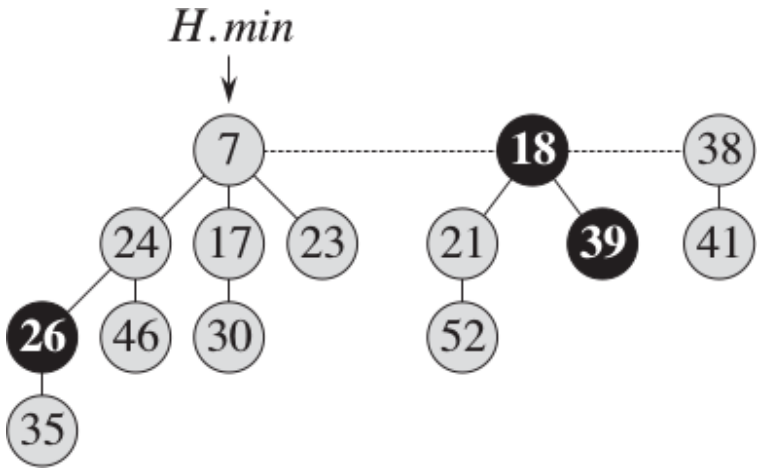
Fibonacci heaps – **consolidate** operation ...continued





- Now, current=(38) has $degree$=1.
- $A[1]$ is empty, so...
- ...get $A[1]$ to point to the root node at current=(38).
- **consolidate** has now completed one full cycle on the doubly linked list. So, consolidation STOPS!

Fibonacci heaps – **consolidate** operation ...continued



- **extract-min** operation (and consolidation) is now complete.
- Note: during the process of cycling through the **root-list** (during consolidation), we can keep track of the minimum root encountered, and update $H.min$.

- **Amortized** complexity is $O(\log(N))$. (Discussed during week 8 lecture on amortized analysis using the potential function method.)
- The size of auxiliary array A is bounded by $O(\log(N))$ – proof will be discussed as part of slide 41.

Fibonacci heaps – **decrease-key** operation

We want to decrease key of any node x in a Fibonacci heap.[§]

- This can be handled in two cases:

  case 1: When this operation does not violate the heap property (slide #35)

  case 2: When it does!

  - We will handle this over subcases, Case 2a (slide #36) and Case 2b (slide #37-39).

[§]Note: as with binary heaps, Fibonacci heaps are inefficient to **search** for any node x; For this reason, **decrease-key** on x assumes a pointer to x as part of its input.

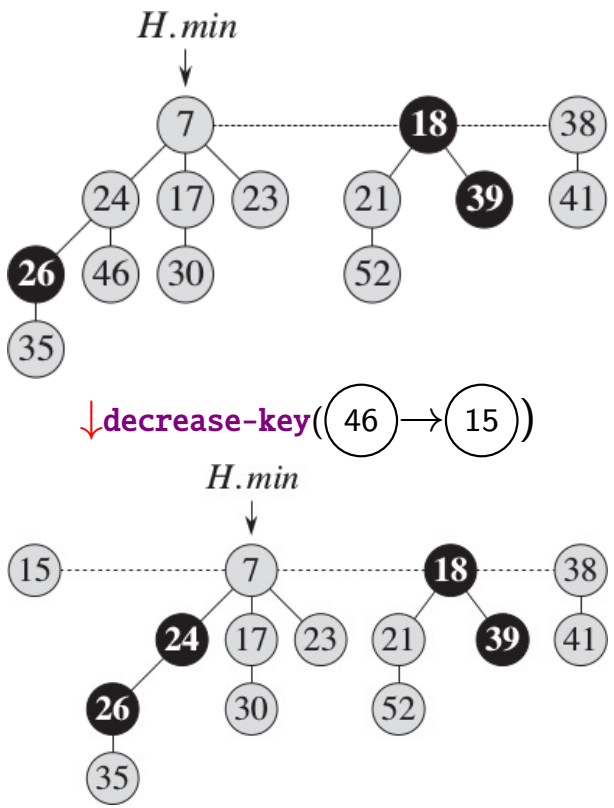
Fibonacci heaps – **decrease-key**: Case 1

When **decrease-key** does not violate the heap property. Simply decrease the key on the node, and we are done!

[§]Note: as with binary heaps, Fibonacci heaps are inefficient to **search** for any node x; For this reason, **decrease-key** on x assumes a pointer to x as part of its input.

Fibonacci heaps – **decrease-key**: Case 2a

Consider an example where we want to **decrease-key** of node with key= 46 to key= 15.

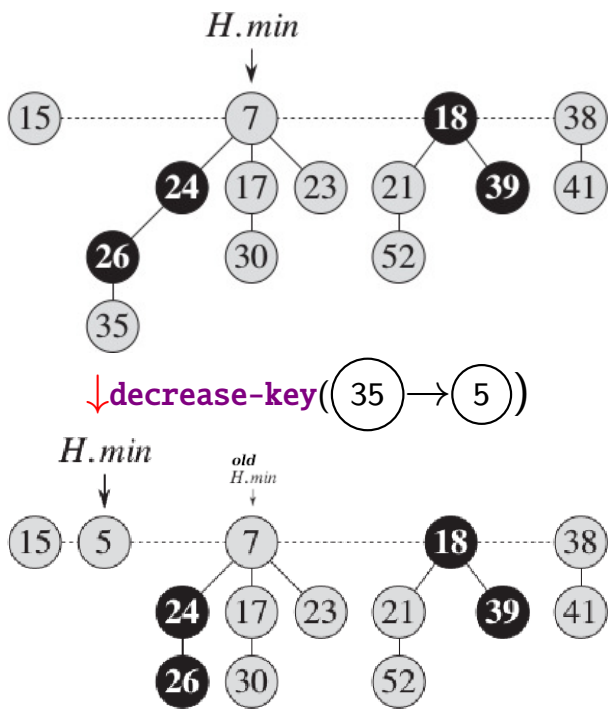


- Decreasing 46 to 15 violates the heap property because its parent 24 > 15.
- To address this violation:
  - Cut the subtree rooted at 15 (prev. 46)...
  - ...and promote it into the root list. (Update $H.min$, if necessary.)
  - If necessary, update the mark of 15 to 'unmarked' after promoting to the root level.
  - Since parent 24 was originally **unmarked** (i.e., hasn't yet lost a child), **mark it** (i.e., has lost a child);

[§]Note: as with binary heaps, Fibonacci heaps are inefficient to **search** for any node x; For this reason, **decrease-key** on x assumes a pointer to x as part of its input.

Fibonacci heaps – **decrease-key**: Case 2b

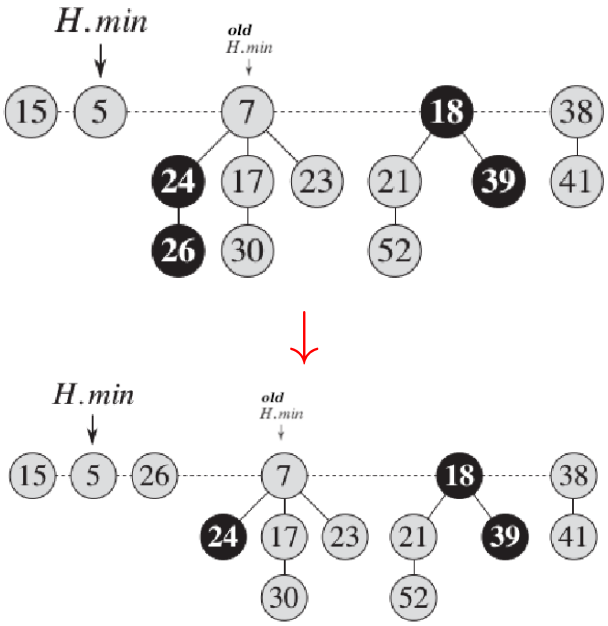
Another case arises, when the original parent of the subtree (promoted to the root level) is already '**marked**' – consider the example where we want to **decrease-key** of node (with key=) 35 to 5.



- Decreasing 35 to 5 violates the heap property...
- ...because its parent 26 > 5 (previously 35).
- To address this violation:
  - Cut the subtree rooted at 5 (prev. 35)...
  - ...and promote it into the root list. (Update $H.min$, if necessary.)
  - If necessary, update the mark of 5 to '**unmarked**' after promoting to the root level.
  - But parent 26 is already '**marked**' (i.e., has lost one child previously);
  - so, repeat this cut-and-promote-to-root process for 26...

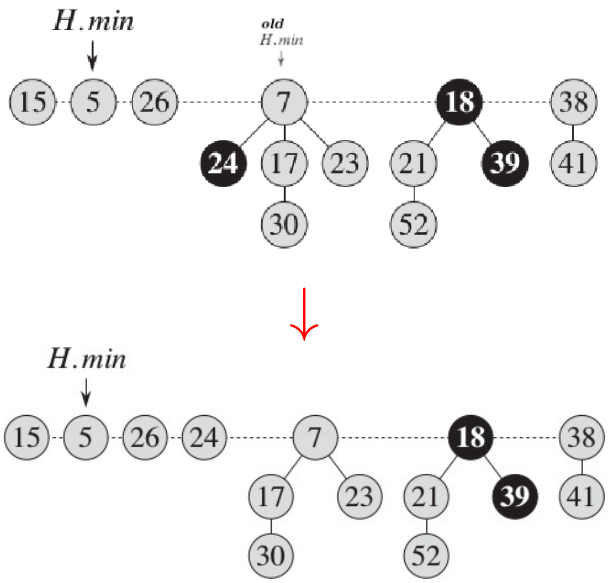
[§]Note: as with binary heaps, Fibonacci heaps are inefficient to **search** for any node x; For this reason, **decrease-key** on x assumes a pointer to x as part of its input.

Fibonacci heaps – **decrease-key**: Case 2b (continued)



- … so, cut the subtree rooted at 26.
- …and promote it into the root list.
- If necessary, update the mark of 26 to '**unmarked**' after promoting to the root level.
- But its parent 24 is again already '**marked**' (i.e., has lost one child previously);
- so, repeat this cut-and-promote-to-root process for 24…

Fibonacci heaps – **decrease-key**: Case 2b (continued)



- ...now, cut the subtree rooted at 24.
- ...and promote it into the root list.
- If necessary, update the mark of 24 to 'unmarked' after promoting to the root level.
- Finally, since its original parent 7 is 'unmarked', STOP!
  - Btw, we do not have to 'mark' a previously unmarked root (when it is a parent of a child that is cut and promoted).

Amortized complexity of **decrease-key** is $O(1)$.

(Discussed during week 8 lecture on amortized analysis using the potential function method.)

Fibonacci heaps – **delete** operations:

delete operation deletes some specified node x. This can be composed using the following two operations, which we already discussed:

- **decrease-key** of x to $-\infty$.
- **extract-min**.

[§]Note: as with binary heaps, Fibonacci heaps are inefficient to **search** for any node x; For this reason, **delete**(x) assumes a pointer to x as part of its input.

So where if **Fibonacci** in all this?

- For a Fibonacci heap, Tarjan (who invented this data structures) showed that, for any node x,
  1. size$(x) \geq F_{\text{degree(x)}+2}$ (F_i denotes the ith Fibonacci number), which implies
  2. degree$(x) \leq \log_\phi(N)$, where $\phi = \frac{1+\sqrt{5}}{2} = 1.618033\ldots$ is the golden ratio.

This proof will be handled during the lecture (also on moodle week 7 section).

Summary of Fibonacci heaps

Operation	Fibonacci heap
min	$O(1)$
extract-min	$O(\log N)$ (amortized)
merge	$O(1)$
decrease-key	$O(1)$ (amortized)
delete	$O(\log N)$ (amortized)
insert	$O(1)$

In the next lecture...

Amortized analysis (the aggregate method, the accounting method and the potential method).

-=o0o=-

END

-=o0o=-